DOSSIER D'ARCHITECTURE LOGICIELLE • CONFIDENTIEL

CRM Architecture Complète

Reverse Engineering & Documentation UML de Niveau Entreprise
Backend Node.js/Express · Frontend Flutter · Base de données PostgreSQL

Use Case | Classes | Séquence | Activité | États | Communication | Packages | Composants | Déploiement | ERD | DDD

Projet analysé : CBI Tunisia — Dash Master Toolkit / Backend Master
Document généré le 27 juillet 2026 · Analyse en lecture seule, aucun code modifié

Table des matières

1. Résumé exécutif

2. Architecture globale

- 2.1 Architecture technique d'ensemble
- 2.2 Diagramme de composants
- 2.3 Diagramme de déploiement
- 2.4 Diagramme de packages
- 2.5 Diagramme des dépendances
- 2.6 Diagramme des flux de données

3. Présentation métier

- 3.1 Acteurs et rôles
- 3.2 Vue d'ensemble des modules métier
- 3.3 Workflows principaux
- 3.4 Règles métier transverses
- 3.5 Événements du domaine
- 3.6 Matrice de permissions
- 3.7 Diagramme de cas d'utilisation

4. Diagramme de classes & diagramme d'objets

- 4.1 Classes du cœur CRM (Projets & Pipeline)
- 4.2 Classes du suivi commercial
- 4.3 Classes des demandes internes
- 4.4 Classes du domaine industriel
- 4.5 Classes d'architecture frontend
- 4.6 Diagramme d'objets

5. Modélisation Domain-Driven Design

- 5.1 Carte des contextes bornés
- 5.2 Entités vs. Value Objects
- 5.3 Agrégats, racines et invariants
- 5.4 Services de domaine
- 5.5 Repositories & Factories
- 5.6 Événements de domaine

6. Diagrammes de séquence

- 6.1 Connexion (avec MFA optionnel)
- 6.2 Rafraîchissement de jeton & déconnexion
- 6.3 Créer un projet
- 6.4 Modifier un projet
- 6.5 Supprimer / Archiver un projet
- 6.6 Créer un client (contact commercial)
- 6.7 Notification (temps réel)
- 6.8 Export PDF (demande RH)
- 6.9 Export Excel (liste de projets)
- 6.10 Gestion des utilisateurs

7. Diagrammes d'activité

7.1 Cycle de vie complet d'un projet

7.2 Demande d'archivage / désarchivage

7.3 Demande RH (congé / sortie)

7.4 Demande de maintenance

7.5 Saisie et validation POR PROMESH

7.6 Suivi commercial et relance

8. Diagrammes d'état

8.1 Projet

8.2 Client (Contact commercial)

8.3 Document (rapport POR PROMESH)

8.4 « Facture » — Devis / Bon de commande

8.5 Demande (RH / Maintenance / Archivage)

8.6 Utilisateur

9. Diagramme de communication

9.1 Traitement d'une demande de maintenance

9.2 Authentification avec garde d'accès

10. Modèle de données (ERD)

10.1 Grappe Identité, sécurité & intégrations

10.2 Grappe Pipeline & Projets

10.3 Grappe Prospection commerciale

10.4 Grappe Workflows d'approbation

10.5 Grappe Opérations industrielles

10.6 Inventaire détaillé des tables

10.7 Tables de jonction & données de référence

10.8 Constats structurels de la base

11. Description module par module

11.1 Pipeline

11.2 Projects

11.3 Project-Actions & Project-Activities

11.4 Kanban

11.5 Commercial-Contacts

11.6 Archive-Requests

11.7 Maintenance-Requests

11.8 HR-Requests

11.9 Industrial-Records

11.10 Por-Promesh

11.11 Recuperables

11.12 Dashboard & Admin-Dashboard

12. Référentiel d'API

13. Glossaire

14. Conclusion & recommandations

1. Résumé exécutif

Ce document est le résultat d'une rétro-ingénierie (reverse engineering) complète et en lecture seule du CRM développé pour **CBI Tunisia** (marques PROBAR / PROMESH). Le système est composé de trois briques : une application **Flutter** multiplateforme (« Dash Master Toolkit »), une **API Node.js/Express** (« Backend Master ») et une base de données **PostgreSQL** pilotée par des migrations Sequelize. Aucune ligne de code n'a été modifiée : toutes les informations proviennent de la lecture exhaustive des routes, contrôleurs, services, modèles, migrations et écrans réellement présents dans le dépôt.

~53 modèles Sequelize / tables logiques	**96+** fichiers de migration PostgreSQL
14 modules métier backend (src/modules)	**402** fichiers Dart (frontend Flutter)
7 rôles utilisateurs distincts	**15** types de diagrammes UML produits

Ce qu'est le CRM

Le système couvre un périmètre plus large qu'un CRM commercial classique : il gère le **pipeline commercial** (prospects, contacts, projets de construction, revendeurs, applicateurs), la **gestion de chantier** (actions, relances, devis, bons de commande, agenda synchronisé avec Google Calendar), les **processus industriels** (fiches de production PROBAR/MÉLANGE, rapports d'équipe POR PROMESH, suivi des récupérables/déchets), ainsi que des **processus internes d'entreprise** (demandes RH de congé/sortie, demandes de maintenance d'équipement, demandes d'archivage de projet). C'est donc un **ERP/CRM hybride** plutôt qu'un CRM pur.

Architecture en un coup d'œil

- **Frontend :** Flutter (Web/Desktop/Mobile), construit sur un template UI-Kit commercial (« Dash Master Toolkit ») dont une partie substantielle (tableaux de bord ecommerce/finance/academic, tables de démonstration) reste un gabarit non branché ; la logique métier réelle vit dans `lib/forms/*`, `lib/services/*` et `lib/providers/*`.
- **Backend :** Express 5 + Sequelize 6 sur PostgreSQL, authentification JWT (access 15 min / refresh 7 j en cookie httpOnly) avec MFA par OTP e-mail (actuellement désactivé via variable d'environnement), autorisation par rôles + listes d'e-mails codées en dur pour certains acteurs métier clés, temps réel via Socket.IO, exports Excel (exceljs) et PDF (pdfkit), notifications multi-canal (base de données + socket + e-mail + WhatsApp optionnel via Twilio).
- **Base de données :** PostgreSQL, ~50 tables, gérée exclusivement par migrations (pas de `sequelize.sync()` en production). Les tables historiques (users, projects, notifications…) n'ont pas de contraintes FK physiques (relations uniquement logiques côté Sequelize), tandis que les modules ajoutés récemment (RH, maintenance, archivage, POR PROMESH, MFA) déclarent des clés étrangères réelles.

Constats majeurs (détaillés au chapitre 14 — Conclusion & Recommandations)

> **Sécurité — critique :** le fichier `Backend Master/.env.example` contient des identifiants réels (mot de passe SMTP, mots de passe super-administrateur, clé de chiffrement des jetons Google, secret MFA) au lieu de valeurs d'exemple. Ces secrets doivent être considérés comme compromis, être régénérés immédiatement et purgés de l'historique Git.

> **Architecture :** deux styles cohabitent côté backend — un style « routes grasses » historique (`src/routes/*`) et un style en couches moderne Route→Contrôleur→Service→Repository (`src/modules/*`) — avec, pour `/projects`, un montage simultané des deux routeurs désambiguïsé par un test de format UUID, ce qui est fragile.

> **Autorisation :** il n'existe pas de véritable moteur RBAC ; l'autorisation combine des noms de rôles et des adresses e-mail codées en dur (root admin, gestionnaire RH, responsable maintenance) directement dans le code source.

> **Duplication frontend :** deux implémentations d'`ApiClient`, deux services de profil utilisateur, deux contrôleurs de notification quasiment identiques, et deux styles de gestion d'état (GetX et Provider) cohabitent sans convention unique.

Objectif et portée du document

Ce dossier fournit : une présentation métier complète (acteurs, rôles, règles, workflows), l'ensemble des diagrammes UML demandés (cas d'utilisation, classes, objets, séquence, activité, état, communication, packages, composants, déploiement), le modèle de données complet (ERD), une lecture Domain-Driven Design (entités, value objects, agrégats, services de domaine, événements, contextes bornés), une description module par module, un référentiel d'API, un glossaire et une conclusion avec recommandations priorisées. La profondeur de chaque section reflète fidèlement ce qui a été trouvé dans le code : aucune fonctionnalité n'a été inventée.

2. Architecture globale

2.1 Architecture technique d'ensemble

Le système suit une architecture client-serveur classique à trois niveaux : un client riche Flutter consommant une API REST + WebSocket Node.js, elle-même adossée à PostgreSQL et à des services externes (Google Calendar, SMTP, Twilio WhatsApp, Metabase, OpenStreetMap/Nominatim). Il n'existe **ni conteneurisation Docker, ni reverse-proxy Nginx, ni cache Redis** dans le code actuel : le backend tourne comme un simple processus Node (`node src/app.js` / `nodemon` en développement). Ce point est important pour la suite du document : le diagramme de déploiement au §2.3 présente donc l'état réel (AS-IS) et une cible recommandée (TO-BE) clairement distinguée.

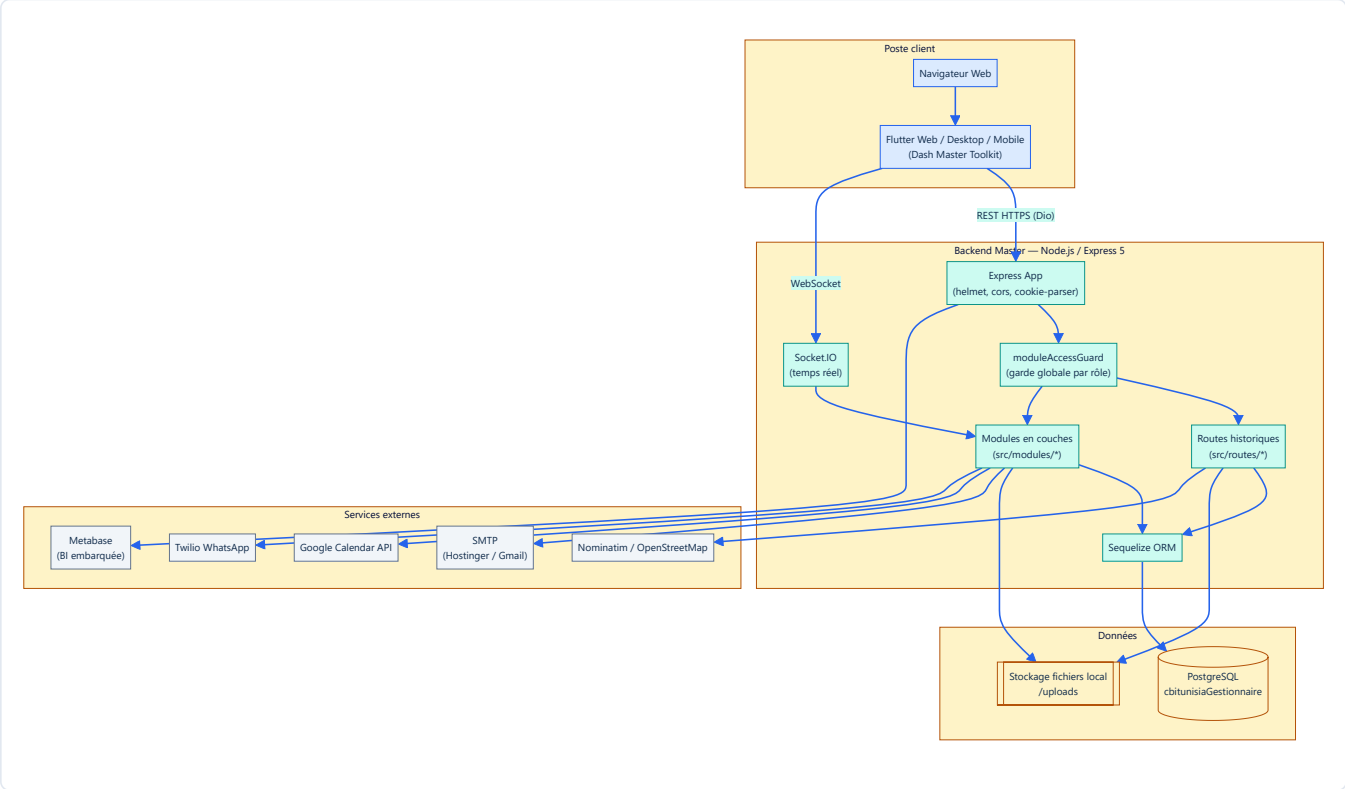


Figure 2.1 — Architecture technique d'ensemble (AS-IS). Légende : bleu = client, turquoise = backend applicatif, ambre = persistance, gris = services tiers. Le flux REST passe systématiquement par la garde `moduleAccessGuard` avant d'atteindre l'un ou l'autre des deux styles de routage coexistants.

2.2 Diagramme de composants

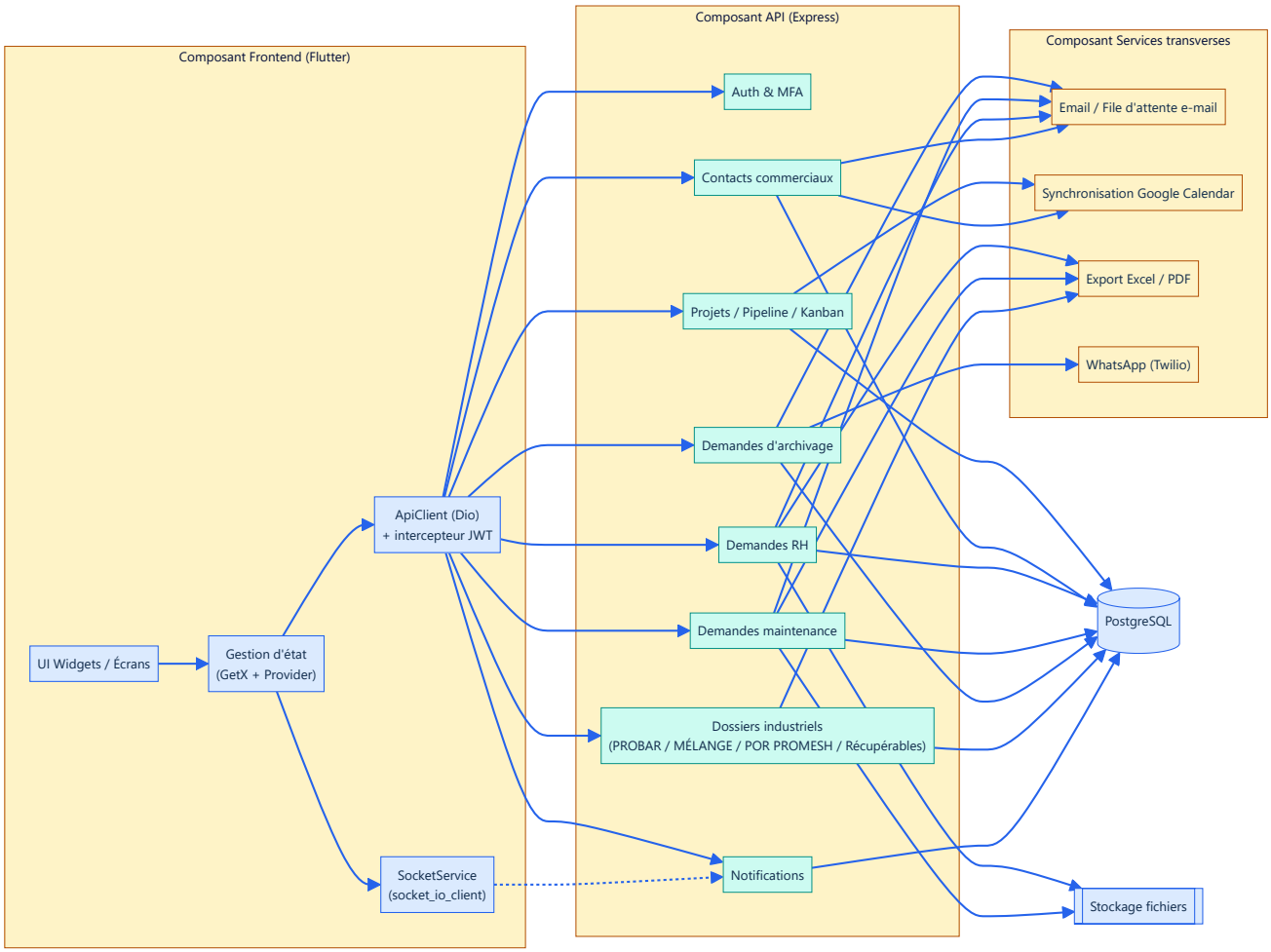


Figure 2.2 — Diagramme de composants. Chaque composant métier backend (Auth, Projets, Contacts, RH, Maintenance, Archivage, Industriel, Notifications) s'appuie sur un socle de services transverses (e-mail, synchronisation calendrier, export, WhatsApp) et persiste dans PostgreSQL.

2.3 Diagramme de déploiement

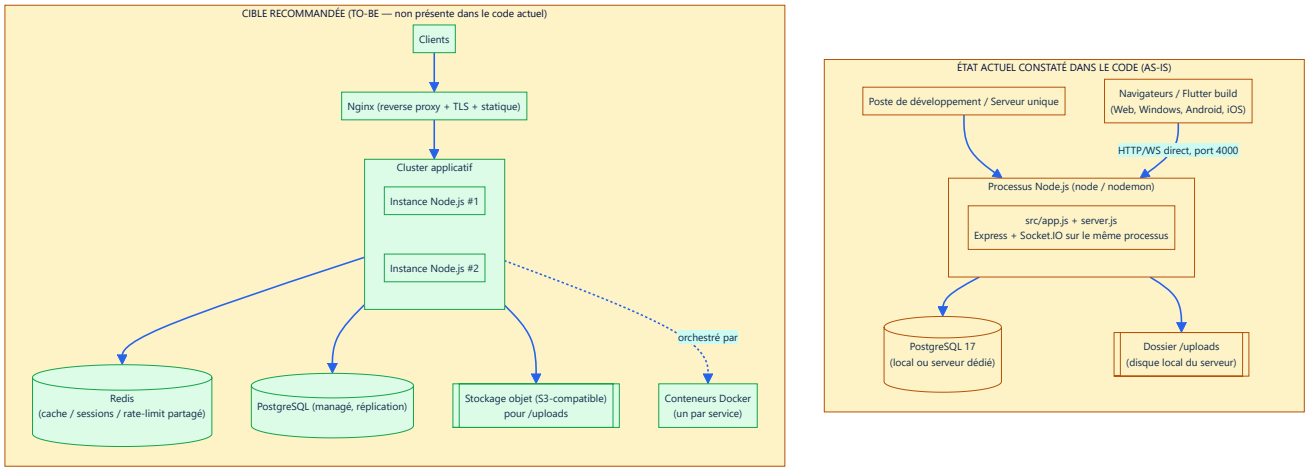


Figure 2.3 — Diagramme de déploiement. Le bloc ambre reflète strictement ce qui existe dans le dépôt (un seul processus Node.js exposant HTTP et WebSocket sur le port `4000`, base PostgreSQL, stockage disque local). Le bloc vert est une cible d'architecture recommandée pour la montée en charge et la haute disponibilité (Nginx, cluster Node, Redis, stockage objet, conteneurisation) — **elle ne correspond à aucun fichier de configuration existant** (aucun Dockerfile, docker-compose ou configuration Nginx n'a été trouvé dans le dépôt).

2.4 Diagramme de packages

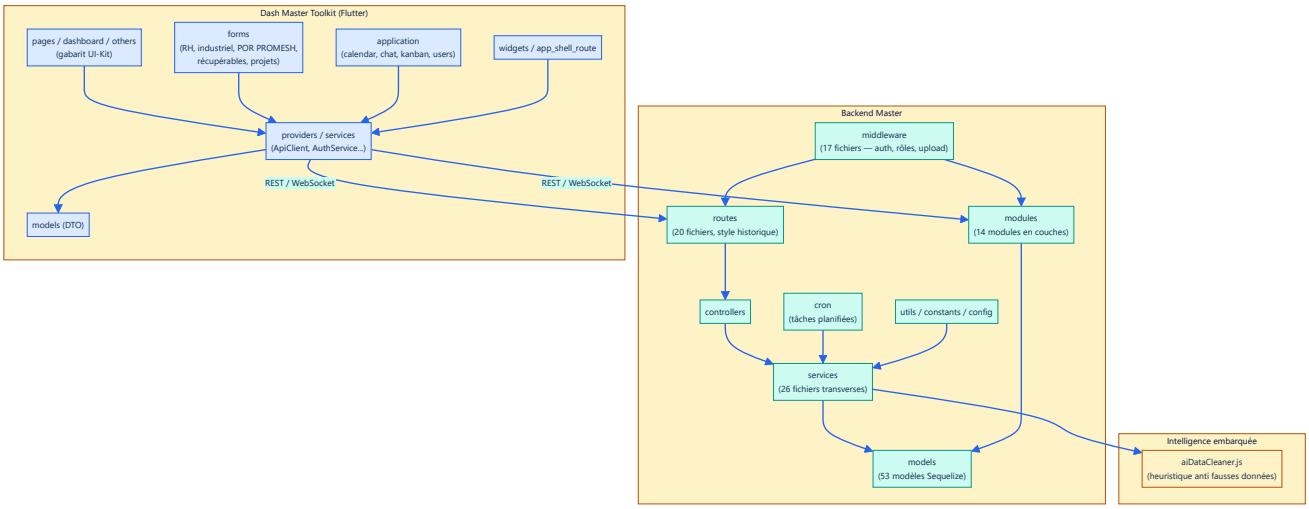


Figure 2.4 — Diagramme de packages. Le seul composant « IA » identifié dans le code est `services/aiDataCleaner.js`, une heuristique de détection de données factices (mots interdits, caractères répétés, absence de voyelles) utilisée lors de l'import de clients — il ne s'agit pas d'un modèle d'apprentissage automatique.

2.5 Diagramme des dépendances

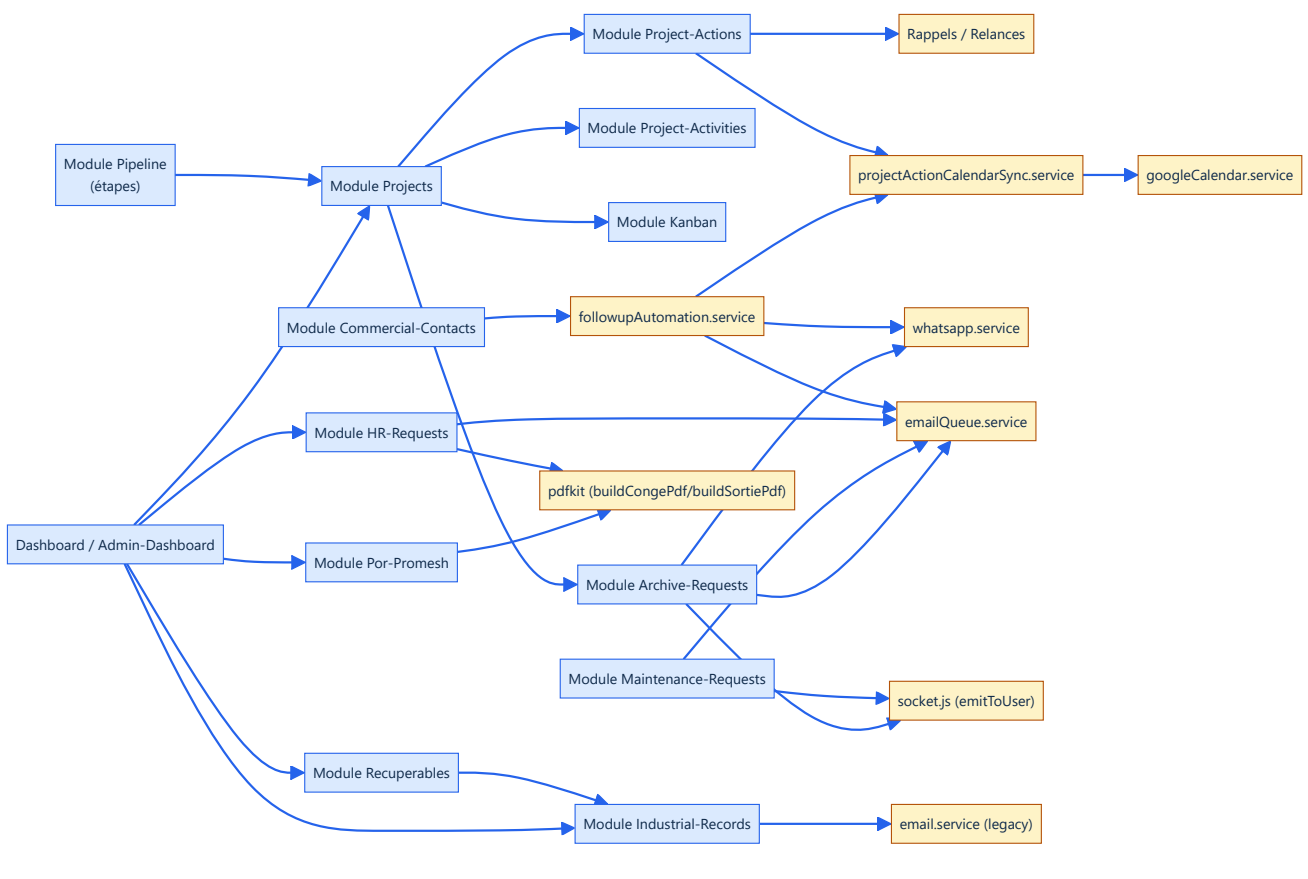


Figure 2.5 — Dépendances inter-modules. Le module **Projects** est le nœud central dont dépendent Pipeline, Actions, Activities, Kanban et Archive-Requests. Le module **Dashboard/Admin-Dashboard** est purement agrégateur (lecture seule) et dépend de presque tous les autres modules métier.

2.6 Diagramme des flux de données

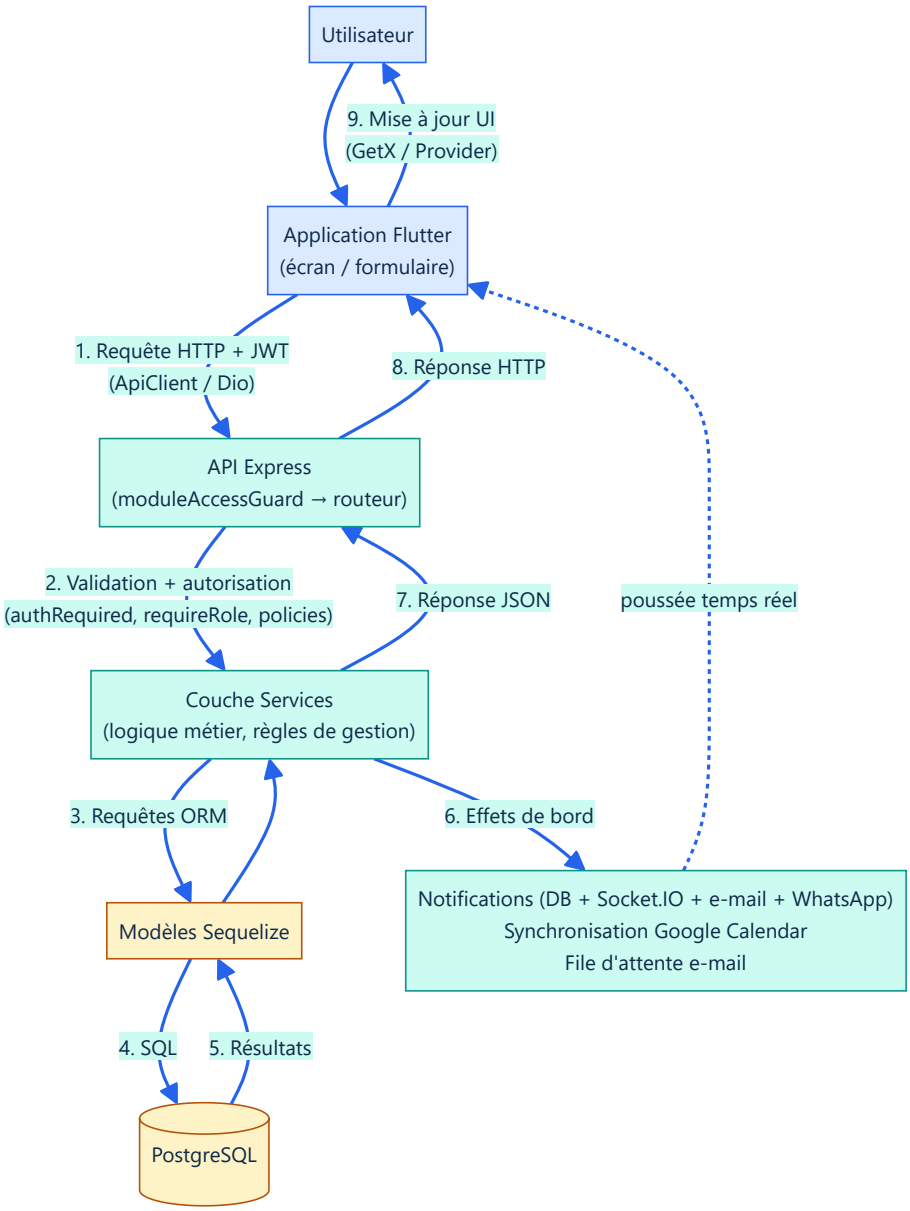


Figure 2.6 — Flux de données de bout en bout. Le retour utilisateur emprunte deux chemins : la réponse HTTP synchrone (étapes 7-8) et, pour les modules à notification (RH, maintenance, archivage, relances commerciales), une poussée asynchrone via Socket.IO qui met à jour l'interface sans rechargement.

3. Présentation métier

Cette section explique le fonctionnement du CRM du point de vue métier : qui l'utilise, ce que chaque rôle peut faire, quels processus il outille et quelles règles de gestion sont réellement appliquées dans le code (et non dans une spécification théorique).

3.1 Acteurs et rôles

Le contrôle d'accès repose sur une colonne `users.role` (ENUM PostgreSQL à 7 valeurs) combinée, pour quatre responsabilités sensibles, à des **adresses e-mail codées en dur** dans la configuration backend (aucune table de permissions ou de délégation configurable n'existe).

Rôle / acteur	Type	Portée & capacités observées dans le code
`user`	Rôle de base	Collaborateur standard : accède à ses propres projets/tâches, peut créer des demandes RH et de maintenance, consulter ses notifications.
`commercial`	Rôle métier	Gère les contacts commerciaux qu'il a créés ou qui lui sont assignés (`commercialId`), les relances, accède à ses propres KPI (`/commercial-contacts/my-kpi`). Peut être forcé, après connexion, de choisir un « profil commercial » (écran `CommercialSelectionScreen`).
`accueil`	Rôle métier	Rôle d'accueil/réception ; assigné automatiquement à la création de compte pour l'adresse `accueilcbif@gmail.com` (qui est également, par e-mail codé en dur, le « responsable maintenance »).
`responsable_logistique_achat`	Rôle métier	Rôle industriel : cantonné par la garde globale `moduleAccessGuard` à un préfixe de routes fixe (POR PROMESH, dossiers industriels, RH, récupérables, maintenance, auth, profil). Sur les dossiers industriels/POR PROMESH/récupérables, portée limitée à ses propres enregistrements (`createdBy`).
`admin` / `superadmin` / `superadmin2`	Rôles d'administration	Groupe « admin-tier » utilisé par la plupart des gardes (`requireRole` , `kpiRoleGuard`) : gestion des utilisateurs, KPI globaux, validation RH/maintenance (en plus des e-mails dédiés), CRUD des étapes de pipeline et types d'action.
Root admin (e-mail codé en dur)	Acteur spécial	`cbitunisia@cbi-tunisia.com` — seul habilité à approuver/rejeter les demandes d'archivage/désarchivage et à archiver directement un projet en court-circuitant ce workflow.
Gestionnaire RH (e-mail codé en dur)	Acteur spécial	`manegerofficecbi@gmail.com` — reçoit et instruit les demandes de congé/sortie, en plus des rôles admin-tier.
Responsable maintenance (e-mail codé en dur)	Acteur spécial	`accueilcbif@gmail.com` — instruit les demandes de maintenance, en plus des rôles admin-tier et `responsable_logistique_achat` .
Demandeur maintenance industrielle (e-mail codé en dur)	Acteur spécial	`responsable_logistique@cbi-tunisia.com` — seul autorisé à créer un dossier industriel de type « maintenance » (distinct du module Demandes de maintenance).
Système / Cron	Acteur non-humain	Tâches planifiées (relance automatique, auto-archivage, rappels de relance, renouvellement des canaux Google Calendar) — désactivées par défaut via `AUTO_EMAIL_JOBS_ENABLED` / `AUTO_ARCHIVE_ENABLED` .
Google Calendar (webhook)	Système externe	Pousse des notifications de changement vers `POST /google-calendar/webhook` pour la synchronisation entrante.

3.2 Vue d'ensemble des modules métier

Module	Objectif métier
Pipeline	Définit les étapes du entonnoir de vente (Prospect → Contacté → Visite → … → Gagné/Perdu) utilisées par Projects et Kanban.
Projects	Cœur du CRM : chantiers, prospects revendeurs/applicateurs, cycle de vie complet (identification → validation → archivage).
Project-Actions / Project-Activities	Journal d'activités commerciales (visites, devis, négociations) et fil d'audit associé à chaque projet.
Kanban	Vue tableau (glisser-déposer) sur les projets et leur étape de pipeline.
Commercial-Contacts	Gestion des prospects/leads, relances, historique de statut, KPI par commercial.
Archive-Requests	Workflow d'approbation pour archiver/désarchiver un projet (le propriétaire demande, le root admin valide).
Maintenance-Requests	Signalement de pannes d'équipement et suivi jusqu'à résolution par un technicien.
HR-Requests	Demandes de congé et d'autorisation de sortie des employés, avec génération de PDF officiel.
Industrial-Records	Fiches de production journalières PROBAR / MÉLANGE / Maintenance (sous-domaine industriel générique).
Por-Promesh	Rapport détaillé d'équipe pour la ligne PROMESH (contrôle machine, contrôle qualité, arrêts, non-conformités), verrouillable après validation.
Recuperables	Suivi des déchets/matières récupérables par machine/ligne/poste/date, fermeture automatique après 6 jours.
Dashboard / Admin-Dashboard	Agrégations et KPI transverses en lecture seule sur l'ensemble des modules ci-dessus.

3.3 Workflows principaux

Cycle de vie d'un projet

Un projet naît en mode `project` , `revendeur` ou `applicateur` (champ `projectModele`), avec des champs obligatoires différents selon le mode. Il progresse dans les étapes du pipeline (par défaut : Prospect → Contacté → Visite → Plan technique → Échantillonnage → Devis envoyé → Négociation → Gagné/Perdu → Fidélisation), chaque action commerciale pouvant faire progresser l'étape et générer un rappel de relance. Un projet inactif peut être archivé — soit directement par le root admin, soit via le workflow d'approbation Archive-Requests pour tout autre utilisateur.

Suivi commercial (Commercial-Contacts)

Un contact est créé avec un statut d'appel (`ok` , `rappeler_plus_tard` , `user_injoignable` , `client_refuse`) et une étape de pipeline propre (distincte de celle des projets). Les relances ne peuvent être créées que pour les statuts `ok` ou `rappeler_plus_tard` . Chaque changement de statut est journalisé de façon immuable (historique d'audit).

Demandes internes (RH, Maintenance, Archivage)

Les trois modules partagent le même schéma de workflow : création par le demandeur → instruction par un rôle habilité (acceptation/rejet, éventuellement affectation à un technicien pour la maintenance) → clôture, avec à chaque transition un triplet notification base de données + Socket.IO + e-mail (et WhatsApp pour l'archivage). Les demandes RH génèrent en plus un PDF officiel (congé ou autorisation de sortie) transmis par e-mail au gestionnaire RH.

Production industrielle (Industrial-Records / Por-Promesh / Recuperables)

Ces trois modules capturent l'activité d'atelier par poste (matin/nuit). Por-Promesh introduit une notion de **verrouillage** : une fiche reste modifiable tant qu'elle est à l'état `BROUILLON` , puis devient définitive et non modifiable après validation (`VALIDE` , `isLocked=true`). Recuperables introduit une **clôture automatique** après 6 jours si personne ne la clôture manuellement.

3.4 Règles métier transverses observées

- Cohérence produit : `productFamily` (PROBAR/PROMESH) et `diameterMm` sont validés ensemble (PROBAR : Ø4–32 mm, PROMESH : Ø4–10 mm).
- Portée par propriétaire (« owner-scoped ») : motif répété identiquement dans Industrial-Records, Por-Promesh et Recuperables — le rôle `responsable_logistique_achat` ne voit/modifie que ses propres enregistrements, les rôles admin-tier voient tout.
- Irréversibilité contrôlée : verrouillage définitif d'une fiche Por-Promesh après validation ; historiques d'audit en écriture seule (jamais de mise à jour ni suppression) pour les statuts de contact commercial et les activités de projet.
- Dérivation automatique de données : 4 des 14 paramètres « Contrôle Process » de Por-Promesh sont recalculés automatiquement à partir des champs « Contrôle Machine » à chaque sauvegarde, pour éviter une double saisie.
- Fenêtres temporelles métier : fermeture automatique d'une fiche Récupérables après 6 jours ; auto-relance d'un projet inactif au-delà de 48 h (si les tâches planifiées sont activées) ; auto-archivage après un seuil paramétrable (30 jours par défaut).
- Garde-fous de suppression : une étape de pipeline utilisée par au moins un projet ne peut pas être supprimée (HTTP 409).

3.5 Événements du domaine

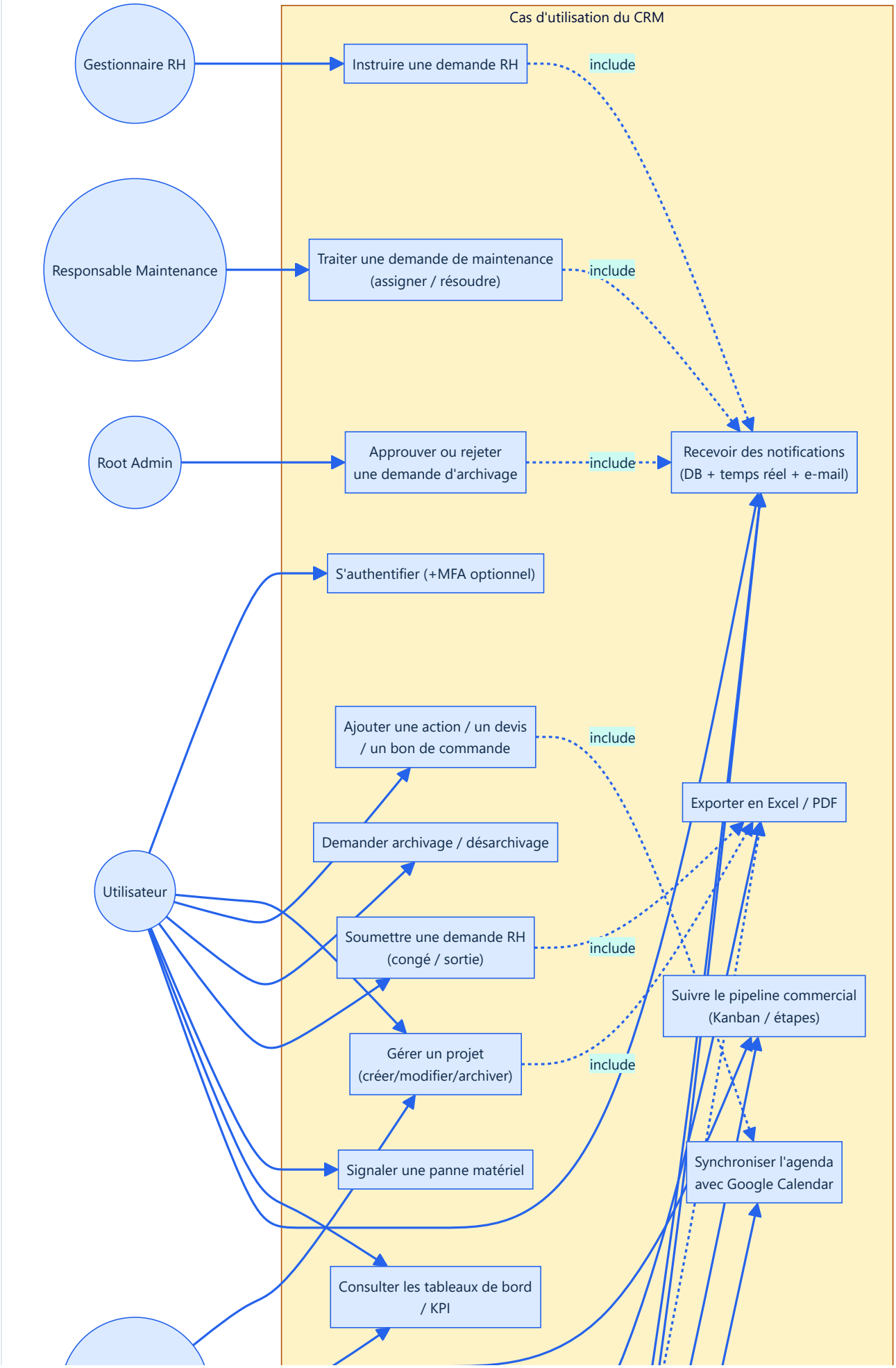
Événement	Déclencheur	Effets observés
ProjectArchiveRequested / Approved / Rejected	Création / décision sur une demande d'archivage	Notification (DB+Socket+e-mail+WhatsApp), écriture dans le journal d'activité du projet, mise à jour de `Project.isArchived` .
MaintenanceRequestStatusChanged	Transition `en_attente→acceptee→en_cours→terminee` (ou `refusee`)	Notification multi-canal, entrée dans `MaintenanceRequestActivity` .
HrRequestReviewed	Acceptation/rejet/traitement d'une demande RH	Génération PDF, e-mail avec pièce jointe au gestionnaire RH, notification au demandeur.
ProjectActionCreated	Ajout d'une action sur un projet	Mise à jour `lastRelanceAt` , création de rappel, synchronisation Google Calendar/CRM (best-effort).
CommercialContactStatusChanged	Changement de `statut` / `pipelineStage`	Écriture dans `CommercialContactStatusHistory` (append-only).
PorPromeshValidated	Appel `POST /por-promesh/:id/validate`	Verrouillage définitif de la fiche (irréversible).
GoogleCalendarChannelExpiring	Cron quotidien 03h00 (Africa/Tunis)	Renouvellement du canal push Google Calendar avant expiration (~30 jours).

3.6 Matrice de permissions (synthèse)

Capacité	user	commercial	accueil	resp. logistique achat	admin-tier (admin/superadmin/superadmin2)	E-mails dédiés
Voir/gérer ses propres projets	✓	✓	✓	—	✓ (tous)	—
Archiver/désarchiver directement	—	—	—	—	—	Root admin uniquement
Approuver demandes d'archivage	—	—	—	—	—	Root admin uniquement
Gérer contacts commerciaux	—	✓ (les siens)	—	—	✓ (tous)	—
KPI commerciaux globaux	—	✓ (les siens)	—	—	✓	—
Instruire demandes RH	—	—	—	—	✓	Gestionnaire RH
Instruire demandes maintenance	—	—	—	✓ (revue)	✓	Responsable maintenance
Créer dossier industriel « maintenance »	—	—	—	—	—	Demandeur maintenance uniquement
CRUD étapes de pipeline / types d'action	—	—	—	—	✓	—

Gestion des utilisateurs (rôles, activation)	—	—	—	—	✓ (activation) / superadmin+ (rôle)	—
POR PROMESH / Industriel / Récupérables	—	—	—	✓ (les siens)	✓ (tous)	—

3.7 Diagramme de cas d'utilisation


Cas d'utilisation du CRM
Gestionnaire RH
Instruire une demande RH
include
Responsable Maintenance
Traiter une demande de maintenance
(assigner / résoudre)
include
Root Admin
Approuver ou rejeter
une demande d'archivage
include
Recevoir des notifications
(DB + temps réel + e-mail)
S'authentifier (+MFA optionnel)
Ajouter une action / un devis
/ un bon de commande
include
Exporter en Excel / PDF
Demander archivage / désarchivage
Utilisateur
Soumettre une demande RH
(congé / sortie)
include
Suivre le pipeline commercial
(Kanban / étapes)
Gérer un projet
(créer/modifier/archiver)
include
Signaler une panne matériel
Synchroniser l'agenda
avec Google Calendar
Consulter les tableaux de bord
/ KPI

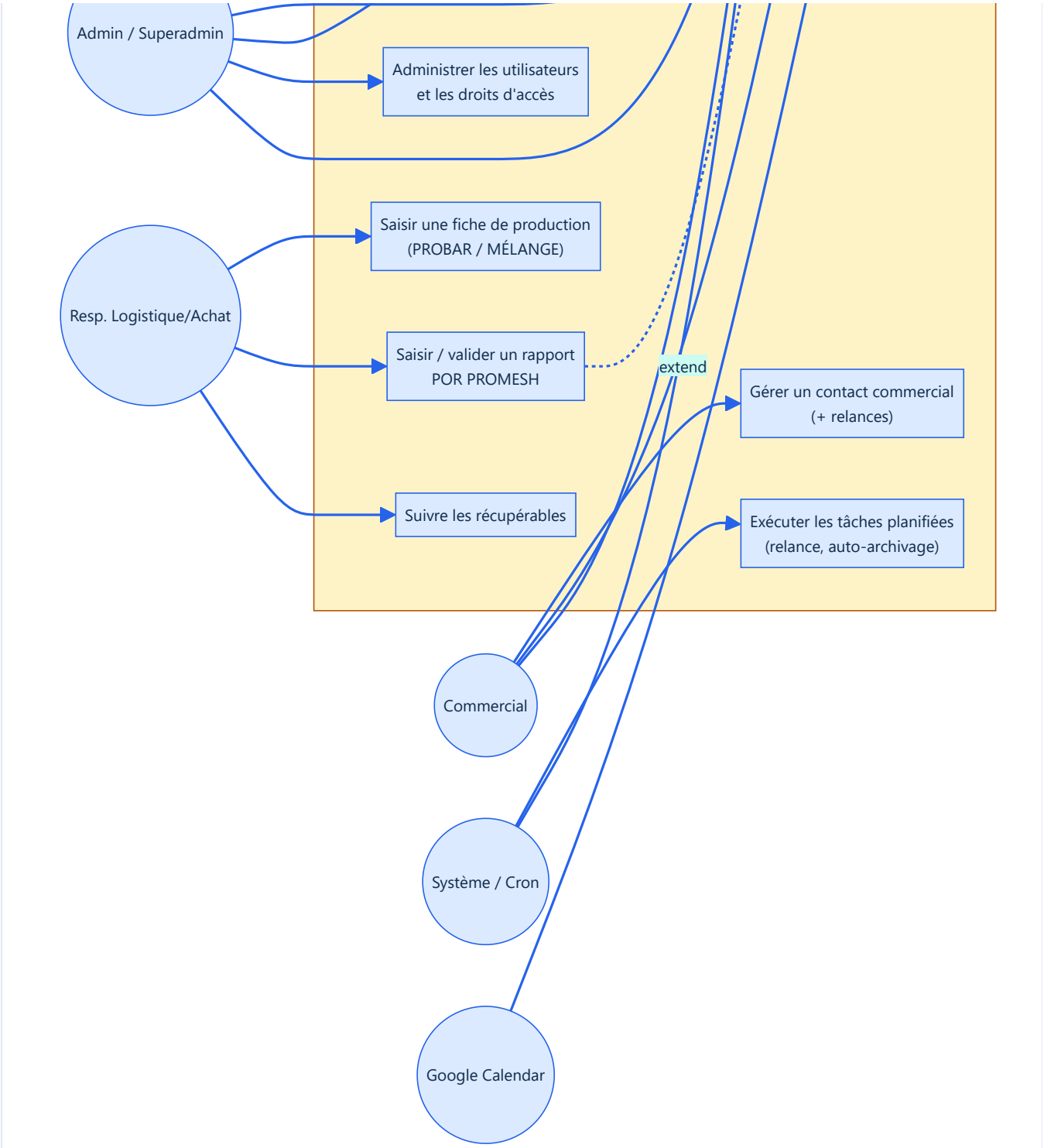


Figure 3.1 — Diagramme de cas d'utilisation. Les relations `include` (pointillé) représentent des sous-fonctions systématiquement déclenchées (ex. : instruire une demande RH inclut toujours l'envoi d'une notification) ; les relations `extend` représentent des comportements optionnels (ex. : l'export PDF n'a lieu que pour certains rapports POR PROMESH).

4. Diagramme de classes & diagramme d'objets

Le backend ne pratique pas l'héritage objet (modèles Sequelize « plats ») : les relations structurantes sont donc des associations, agrégations et compositions. Pour rester lisible, le modèle de classes complet (53 entités) est présenté **par contexte métier** plutôt que dans un unique diagramme monolithique ; le chapitre 10 (ERD) fournit la vue exhaustive colonne par colonne.

4.1 Classes du cœur CRM (Projets & Pipeline)

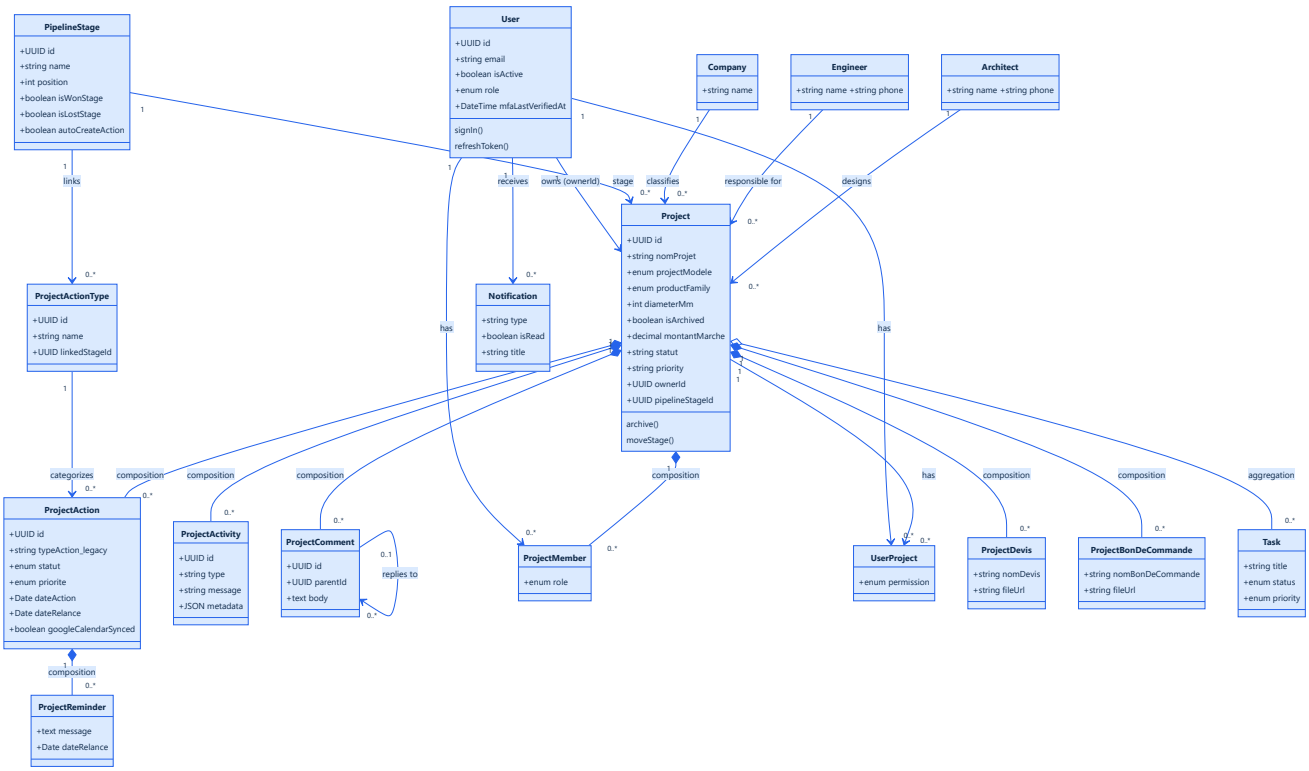


Figure 4.1 — Classes du cœur CRM. Les losanges pleins (`*--`) marquent une composition : la suppression du `Project` entraîne celle de ses actions, activités, commentaires, devis et bons de commande (cascade). Le losange vide (`o--`) entre `Project` et `Task` est une agrégation plus lâche (la tâche calendaire peut survivre indépendamment).

4.2 Classes du suivi commercial (Commercial-Contacts)

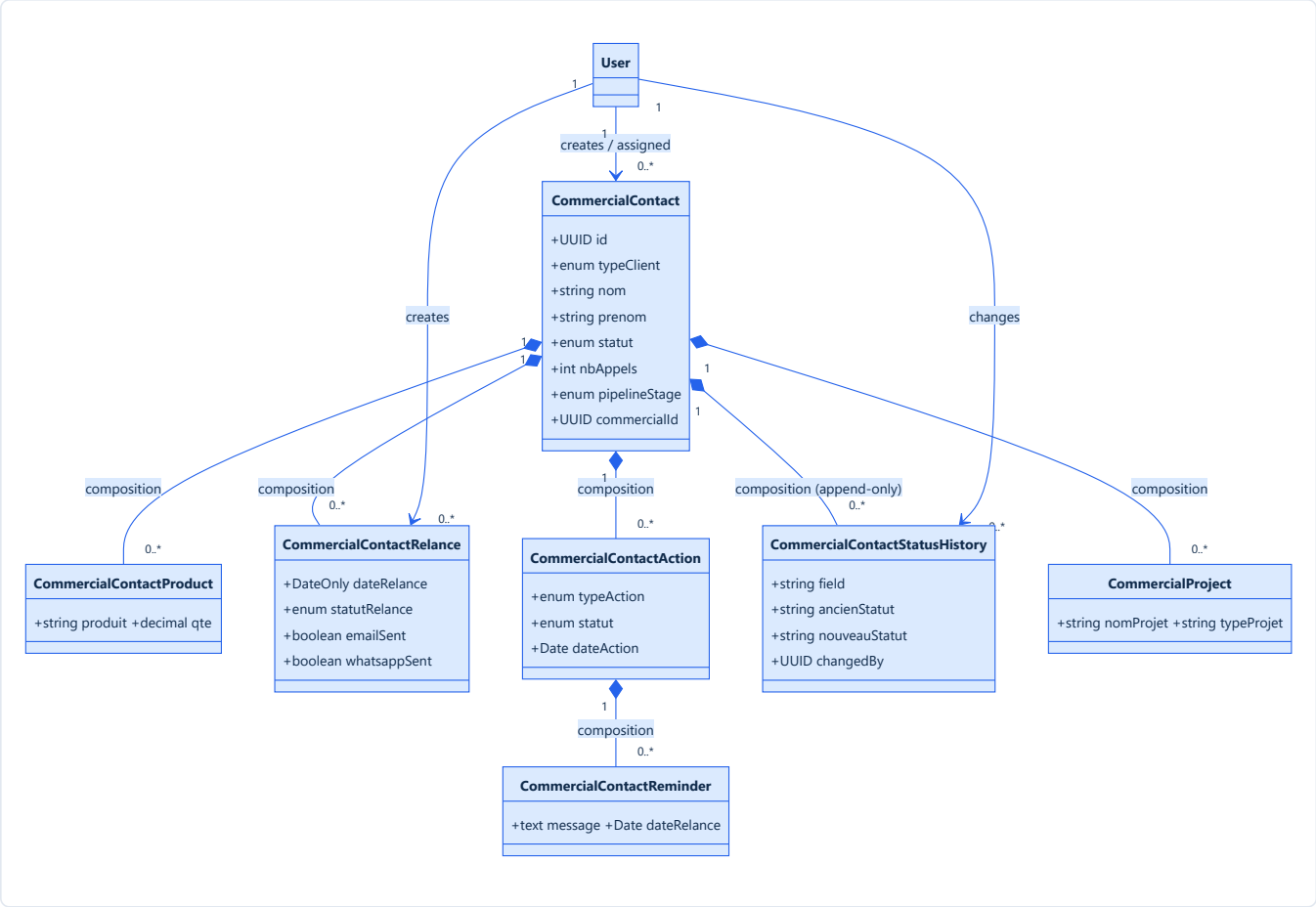


Figure 4.2 — Classes du suivi commercial. `CommercialContactStatusHistory` est un journal d'audit strictement en écriture seule (jamais de update/delete observé dans le code).

4.3 Classes des demandes internes (RH, Maintenance, Archivage)

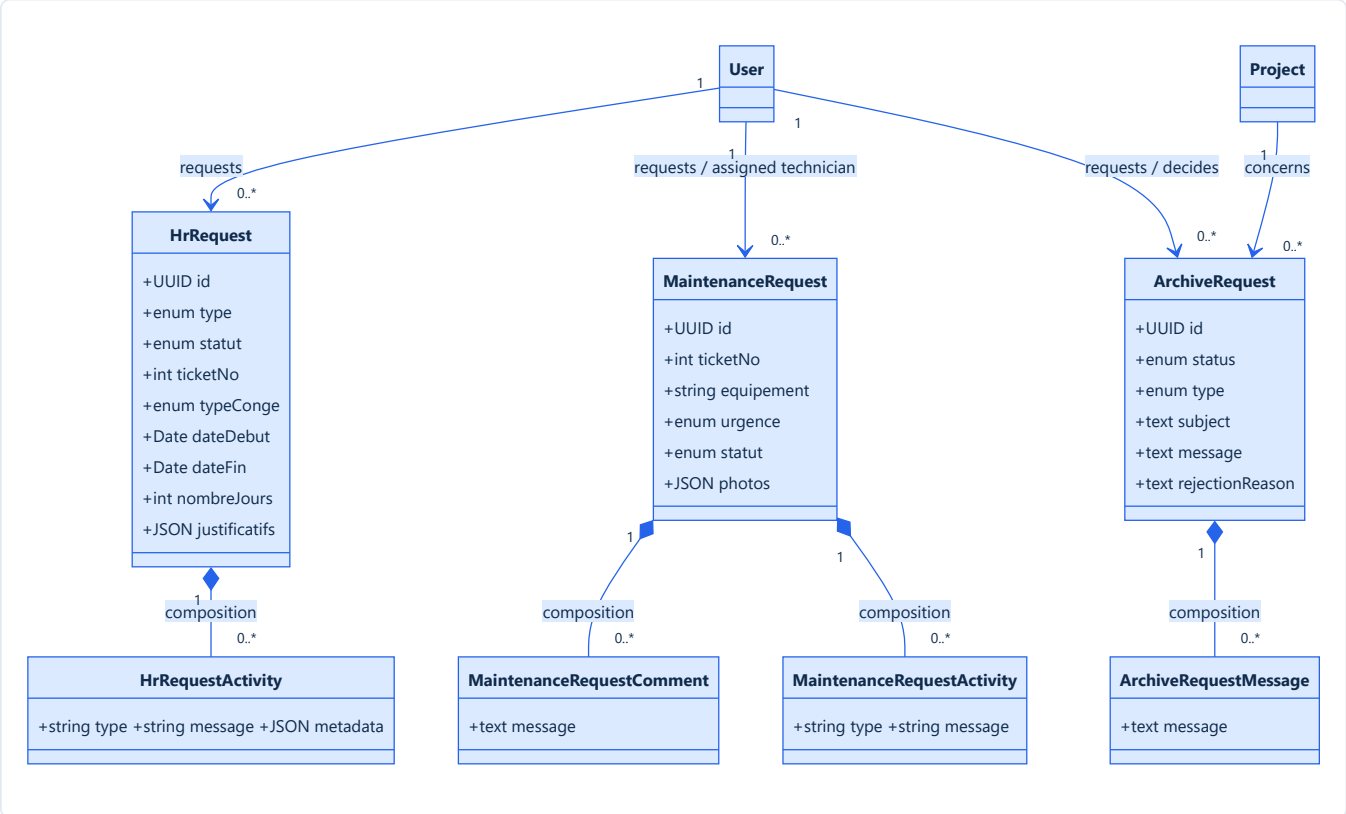


Figure 4.3 — Classes des workflows de demande. Les trois entités `HrRequest`, `MaintenanceRequest` et `ArchiveRequest` partagent la même forme (statut, acteur demandeur, acteur décideur, horodatages de décision) bien qu'elles soient implémentées indépendamment — un candidat naturel à une classe abstraite `ApprovalRequest` non factorisée dans le code actuel.

4.4 Classes du domaine industriel

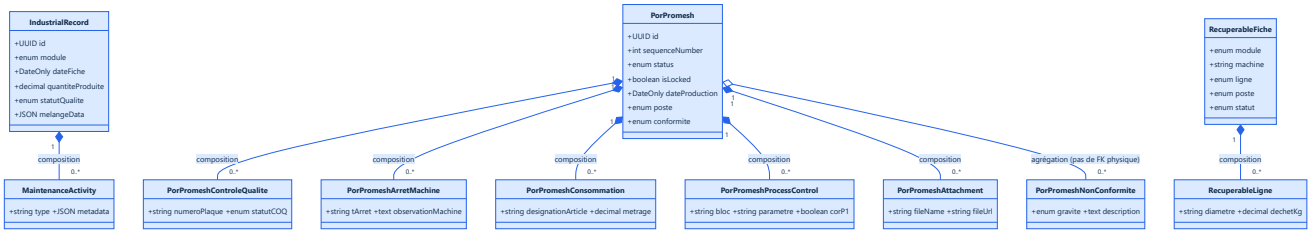


Figure 4.4 — Classes du domaine industriel. `PorPromeshNonConformite` est représentée en agrégation (et non composition) car, à la différence de ses tables sœurs, sa clé étrangère `porPromeshId` n'est pas déclarée au niveau base de données (incohérence relevée au chapitre 10).

4.5 Classes d'architecture frontend (Flutter)

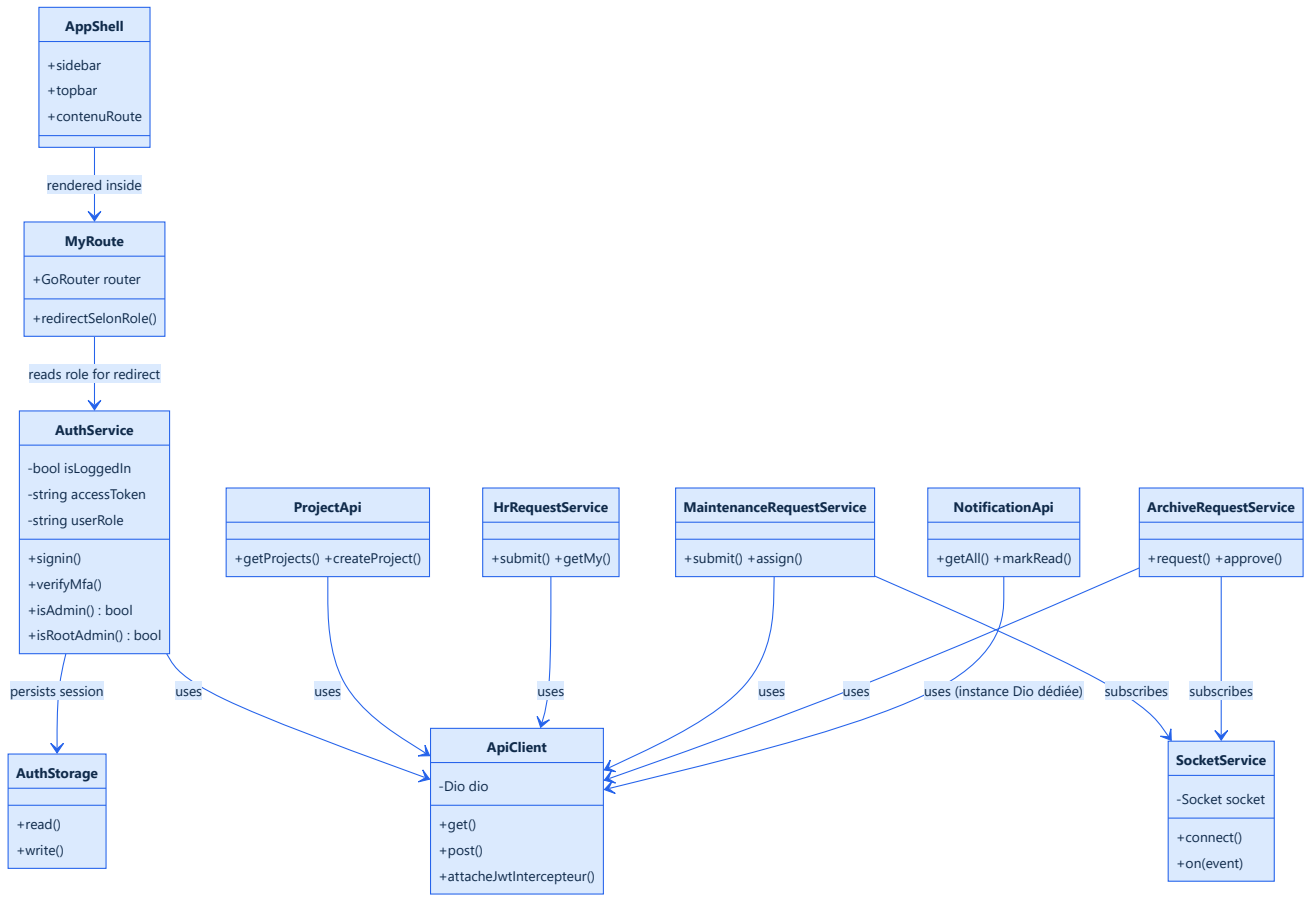


Figure 4.5 — Classes d'architecture frontend. Le diagramme reflète l'implémentation réellement branchée au backend (les écrans « gabarit » du UI-Kit ne sont pas représentés car ils ne consomment aucune classe de service). Une deuxième paire `ApiClient` / `ProjectApi` existe en doublon ailleurs dans le code (voir §8.8).

4.6 Diagramme d'objets (instance illustrative)

Le diagramme suivant montre un instantané d'exécution plausible : un projet en cours de négociation, son propriétaire, son étape de pipeline et sa dernière action commerciale.

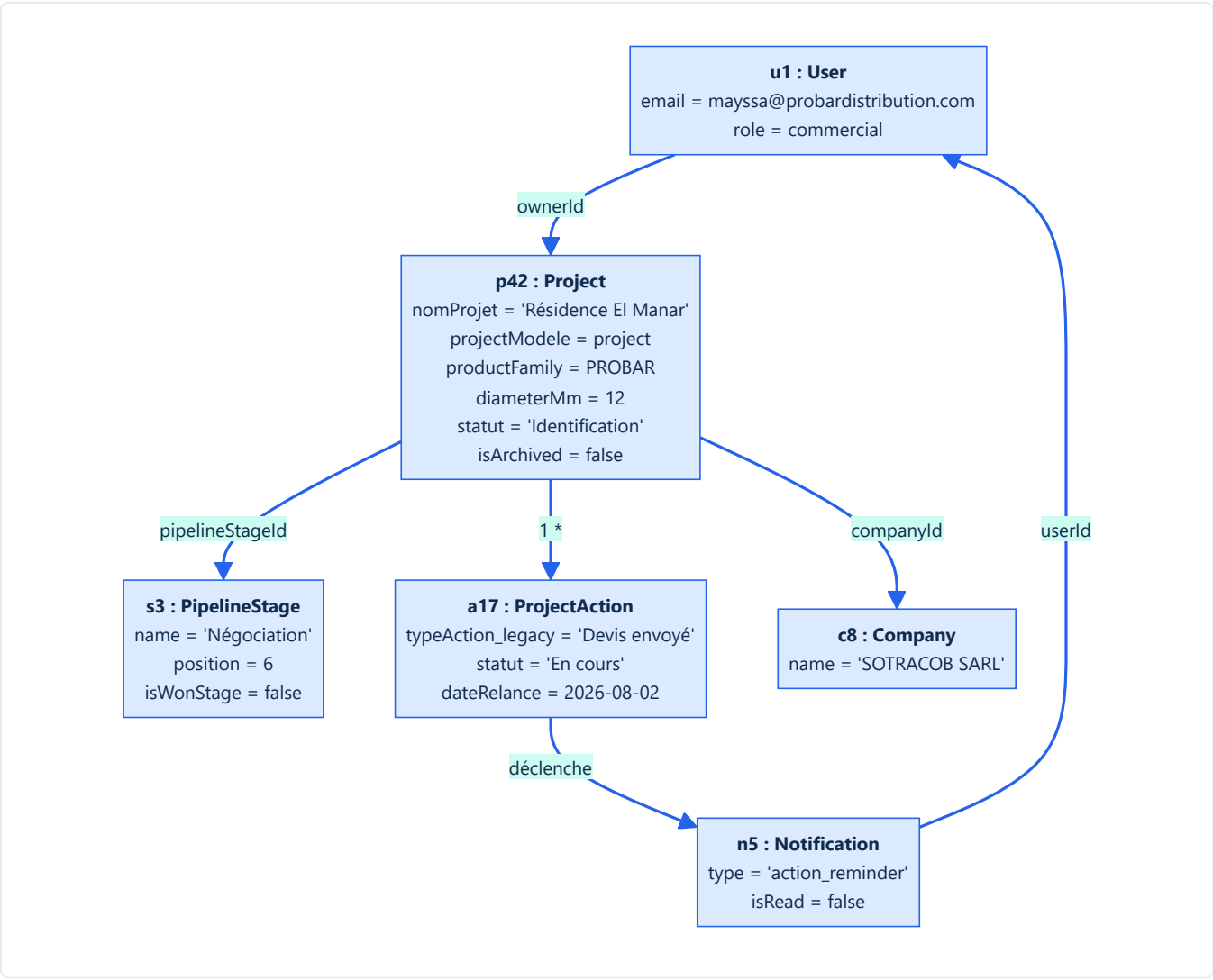


Figure 4.6 — Diagramme d'objets. Instance concrète cohérente avec le modèle de classes du §4.1 : le commercial `u1` possède le projet `p42`, actuellement à l'étape « Négociation », dont la dernière action génère une notification de rappel adressée au même utilisateur.

5. Modélisation Domain-Driven Design

Le code n'a pas été écrit avec un vocabulaire DDD explicite (pas de dossier `domain/`, pas de classes `Aggregate` / `ValueObject` nommées comme telles). Cette section propose donc une **lecture DDD a posteriori** du modèle existant : elle regroupe les entités par contexte borné cohérent et identifie les concepts DDD réellement observables dans le comportement du code (invariants, cascades, services sans état, événements).

5.1 Carte des contextes bornés

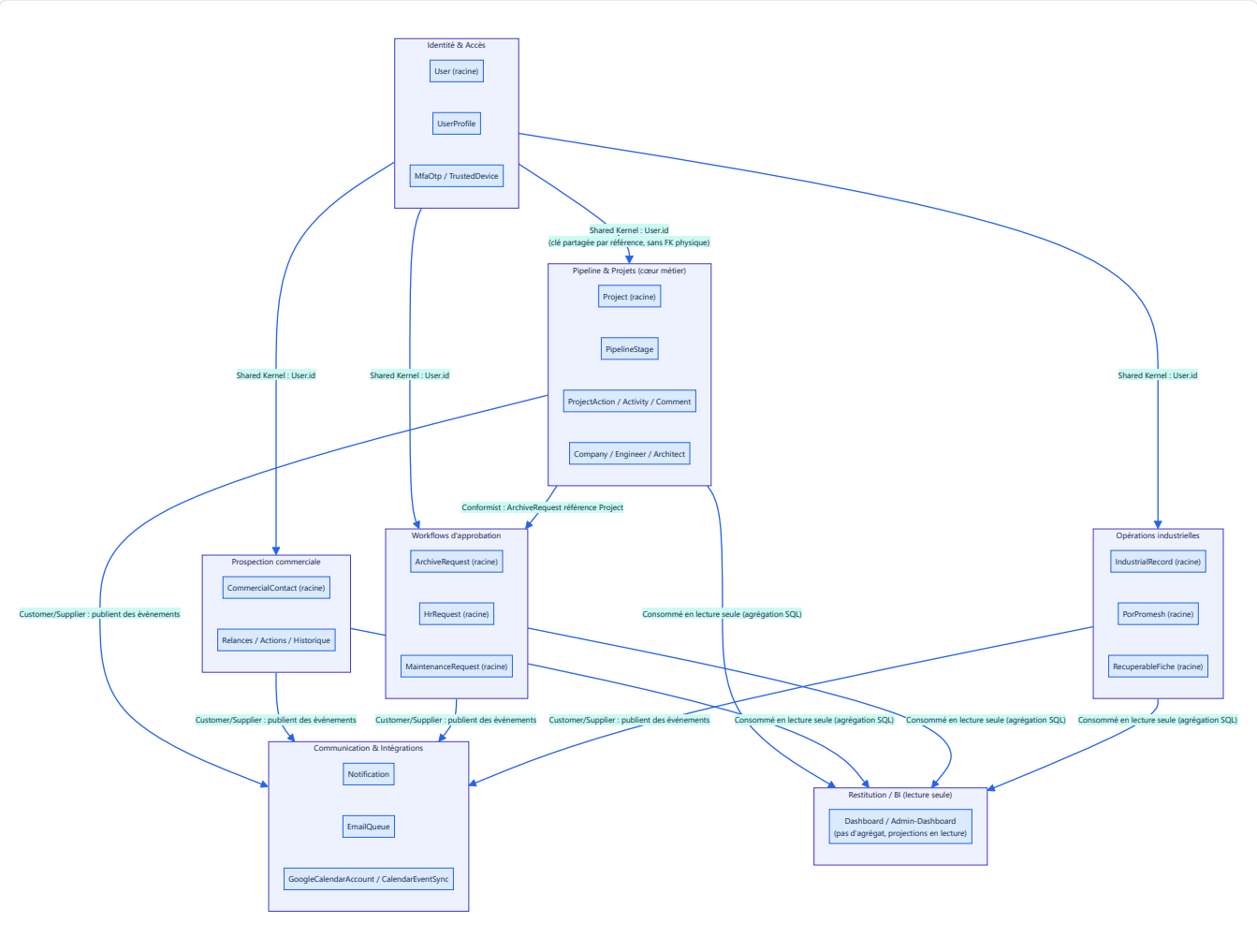


Figure 5.1 — Carte des contextes bornés (Context Map). Le contexte **Identité & Accès** agit comme un *Shared Kernel* minimal (uniquement `User.id`, sans clé étrangère physique dans la plupart des tables historiques). Le contexte **Restitution/BI** n'est pas un contexte métier autonome mais une couche de projection en lecture seule sur les autres.

5.2 Entités vs. Value Objects

La quasi-totalité des tables sont des **Entités** (identité stable par UUID, cycle de vie propre). Peu de *Value Objects* explicites existent en tant que classes séparées ; plusieurs groupes de champs se comportent néanmoins comme des VO conceptuels au sein d'une entité :

Value Object conceptuel	Champs porteurs	Entité hôte	Invariant observé
Spécification produit	`productFamily` + `diameterMm`	Project	Validés ensemble : PROBAR Ø4-32mm, PROMESH Ø4-10mm (jamais l'un sans l'autre).
Géolocalisation	`latitude` + `longitude` + `localisationCommentaire`	Project	Normalisées/validées par `location.service.js` avant écriture.
Période de congé	`dateDebut` + `dateFin` + `nombreJours`	HrRequest	`nombreJours` recalculé en jours ouvrés (lun-ven, sans calendrier de jours fériés).
Fenêtre de service (poste)	`machine` + `ligne` + `poste` + `date`	RecuperableFiche	Clé combinée unique : une seule fiche par combinaison (upsert).

Identité employé figée (snapshot)	`employeeNom/Prenom/Matricule/Qualification/Departement/Service`	HrRequest	Copie immuable du profil au moment de la demande (ne suit pas les modifications ultérieures du profil).
Jeton de session	hash OTP / refresh token / reset token	User, MfaOtp	Toujours stocké sous forme de hachage (SHA-256/bcrypt), jamais en clair.

5.3 Agrégats, racines et invariants

Agrégat (racine)	Entités internes	Invariant garanti par la racine
Project	ProjectAction, ProjectActivity, ProjectComment, ProjectDevis, ProjectBonDeCommande, ProjectReminder, ProjectMember, UserProject	Un projet archivé ne peut être désarchivé que via le workflow d'approbation (sauf root admin) ; la suppression du projet supprime toutes ses entités internes (cascade).
CommercialContact	CommercialContactProduct, Relance, Action, Reminder, StatusHistory, CommercialProject	Une relance ne peut être créée que si `statut` ∈ {ok, rappeler_plus_tard} ; l'historique de statut est en écriture seule.
PorPromesh	ControleQualite, ArretMachine, Consommation, ProcessControl, Attachment, NonConformite	Une fois `status=VALIDE` et `isLocked=true`, plus aucune écriture n'est acceptée sur la racine ni ses enfants.
RecuperableFiche	RecuperableLigne	Fermeture automatique (`statut=cloturee`) au-delà de 6 jours ou sur action explicite « Terminer ».
ArchiveRequest	ArchiveRequestMessage	Un seul type de demande (ARCHIVAGE/DÉSARCHIVAGE) peut être « pending » à la fois pour un même projet.
HrRequest	HrRequestActivity	Transition `traitee` uniquement accessible depuis `acceptee`.
MaintenanceRequest	MaintenanceRequestComment, MaintenanceRequestActivity	Séquence stricte `en_attente→acceptee→en_cours→terminee` (ou `refusee` depuis `en_attente`).
User	UserProfile, MfaOtp, TrustedDevice	Un OTP a un maximum de 5 tentatives et une durée de vie de 10 minutes ; un compte désactivé (`isActive=false`) ne peut pas s'authentifier.

5.4 Services de domaine (sans état)

Service	Rôle
`followupAutomation.service`	Orchestration multi-canal (agenda + notification + e-mail + WhatsApp) pour les relances commerciales automatisées.
`projectActionCalendarSync.service`	Traduction d'une action de projet en événement de calendrier CRM/Google, résolution du propriétaire, re-synchronisation.
`mfa.service`	Génération/validation d'OTP, gestion des appareils de confiance, calcul de l'intervalle de re-vérification selon le rôle.
`companySync.service` / `personSync.service`	Réconciliation des champs libres historiques (`entreprise` , `ingenieurResponsable` , `architecte`) vers des entités `Company` / `Engineer` / `Architect` normalisées.
`emailQueue.service`	File d'attente persistante avec retries exponentiels (30s/1m/2m/5m, jusqu'à 5 tentatives) — garantit la livraison best-effort des e-mails transactionnels.
`aiDataCleaner.js`	Heuristique de qualité de données (détection de faux noms) appliquée lors de l'import de clients.

5.5 Repositories & Factories

Les modules récents (`src/modules/*`) matérialisent explicitement un répertoire `repositories/` par module (hr-requests, maintenance-requests, archive-requests, industrial-records, por-promesh, recuperables, projects, project-actions, pipeline, kanban, dashboard, admin-dashboard) qui encapsule les requêtes Sequelize derrière une interface métier. Les routes historiques (`src/routes/*`) n'ont pas cette séparation : le contrôleur interroge directement les modèles.

Deux « factories » métier identifiables :

- **Auto-provisionnement de type d'action** : la création d'une `PipelineStage` avec `autoCreateAction=true` fabrique automatiquement un `ProjectActionType` lié (« Action - {étape} »).
- **Bootstrap de compte administrateur** : `ensureSuperadmin2()` , exécuté à chaque démarrage du serveur, crée/répare le compte superadmin2 et auto-guérit une valeur ENUM PostgreSQL manquante si nécessaire.

5.6 Événements de domaine

Voir le tableau complet au §3.5. Du point de vue DDD, ces événements sont actuellement **diffusés de façon impérative** (appels directs aux services de notification/e-mail/socket depuis la couche service) plutôt que via un bus d'événements ou un mécanisme de publication/abonnement découplé : il n'existe pas d' `EventEmitter` applicatif ni de file de messages (Kafka/RabbitMQ) dans le dépôt.

6. Diagrammes de séquence

Dix scénarios représentatifs, reconstitués à partir des chemins de code réellement empruntés (middlewares, contrôleurs, services, effets de bord).

6.1 Authentification — Connexion (avec MFA optionnel)

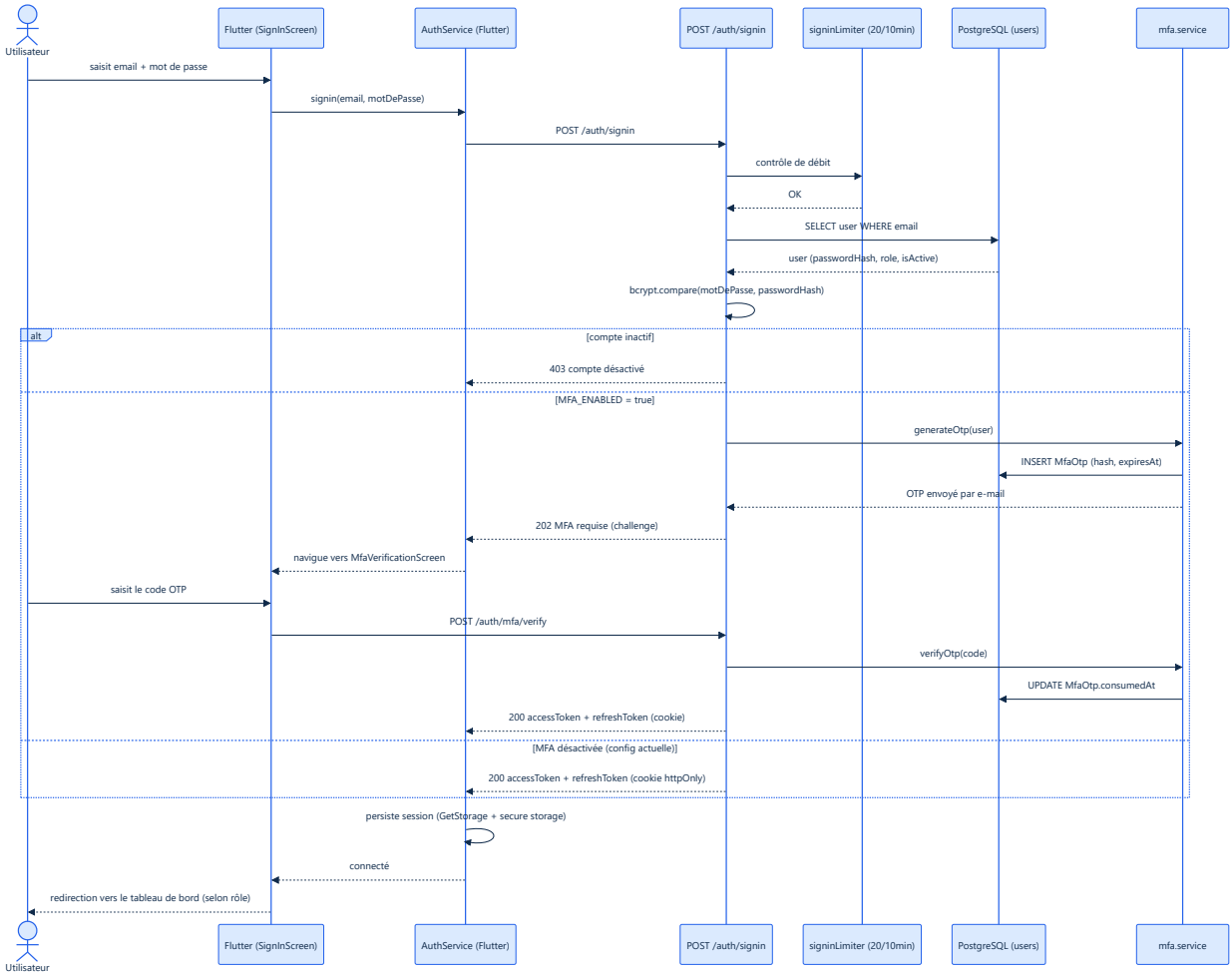


Figure 6.1. Dans la configuration actuelle (`MFA_ENABLED=false`), la branche MFA est ignorée et la connexion aboutit directement à l'émission des jetons.

6.2 Authentification — Rafraîchissement de jeton & déconnexion forcée

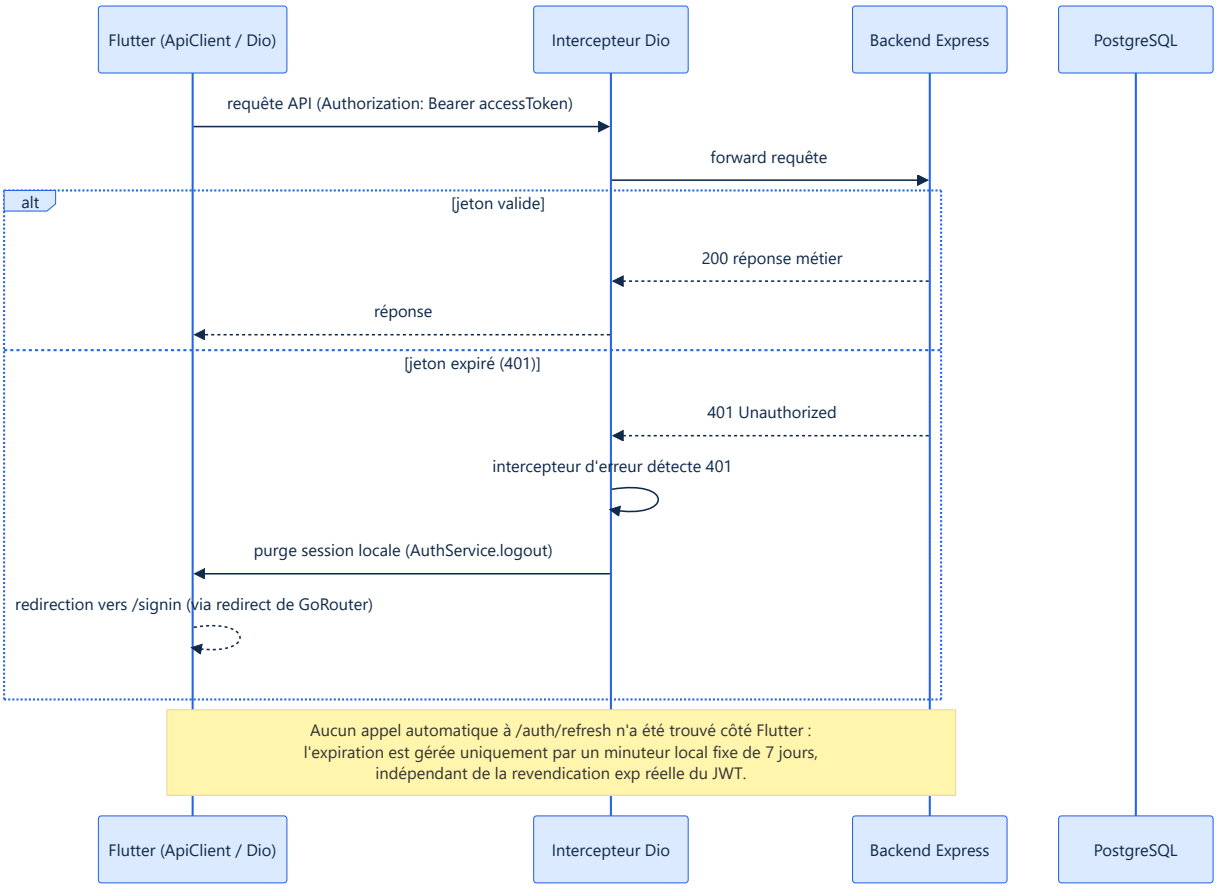
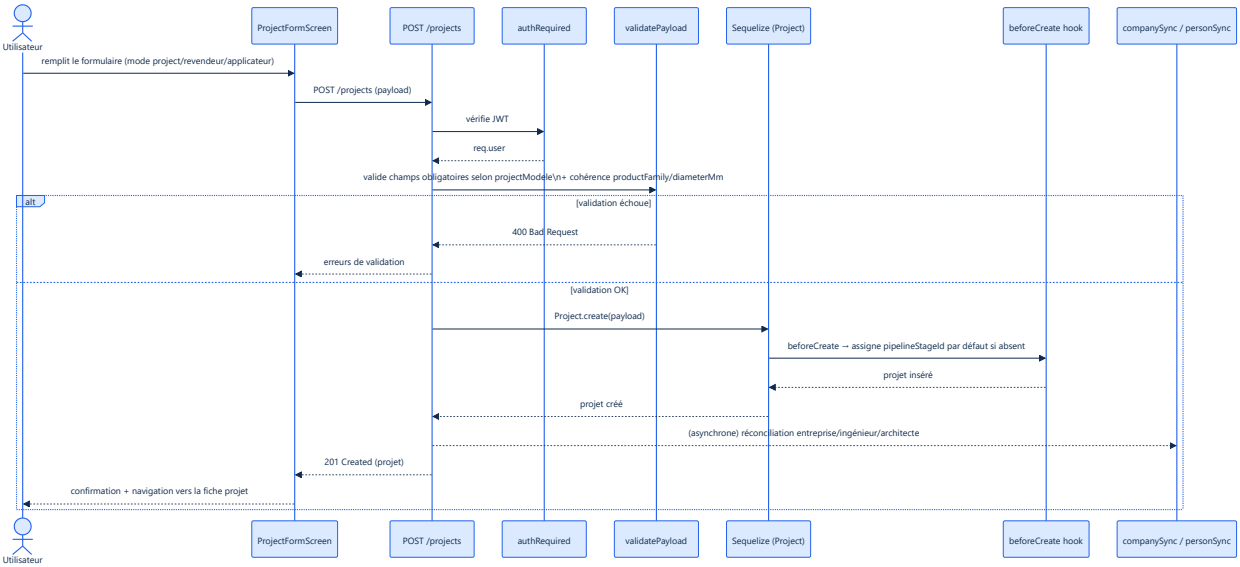
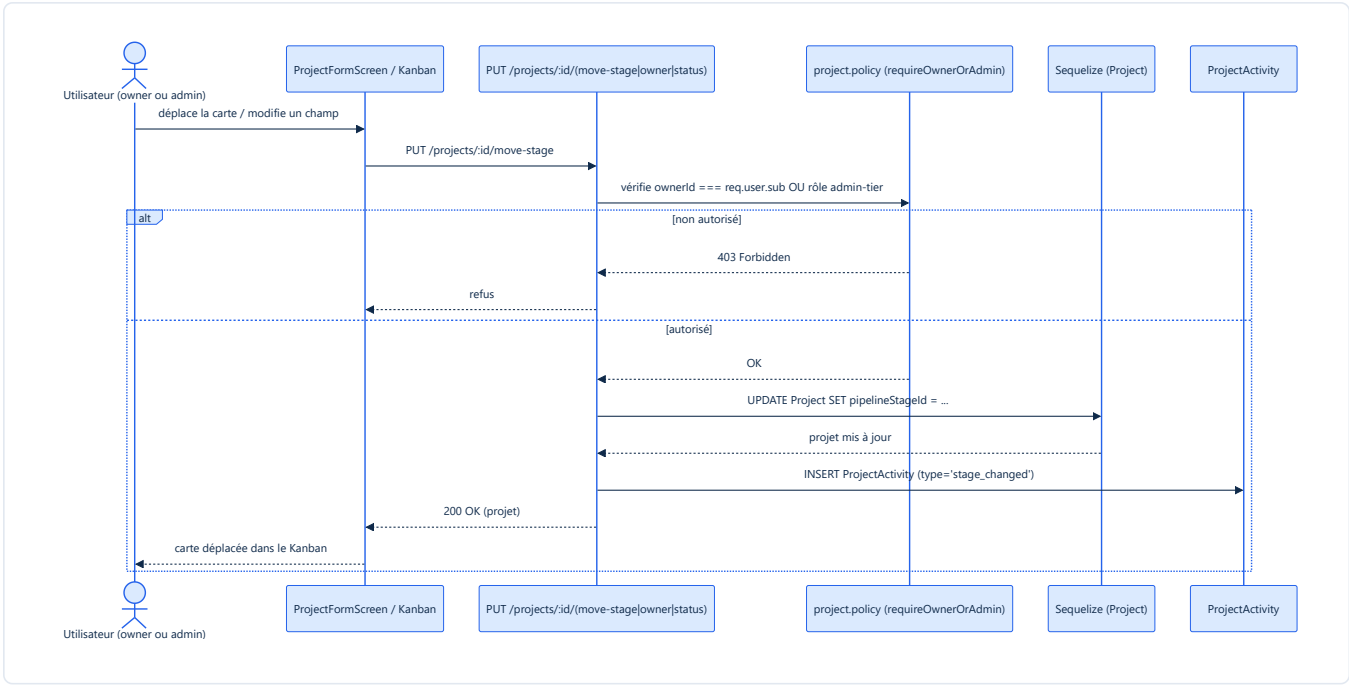


Figure 6.2. Le endpoint `POST /auth/refresh` existe côté backend mais n'est jamais appelé par le client Flutter observé : toute expiration de jeton se traduit aujourd'hui par une déconnexion brutale plutôt qu'un rafraîchissement silencieux.

6.3 Créer un projet



6.4 Modifier un projet (changement d'étape / propriétaire / statut)



6.5 Supprimer / Archiver un projet

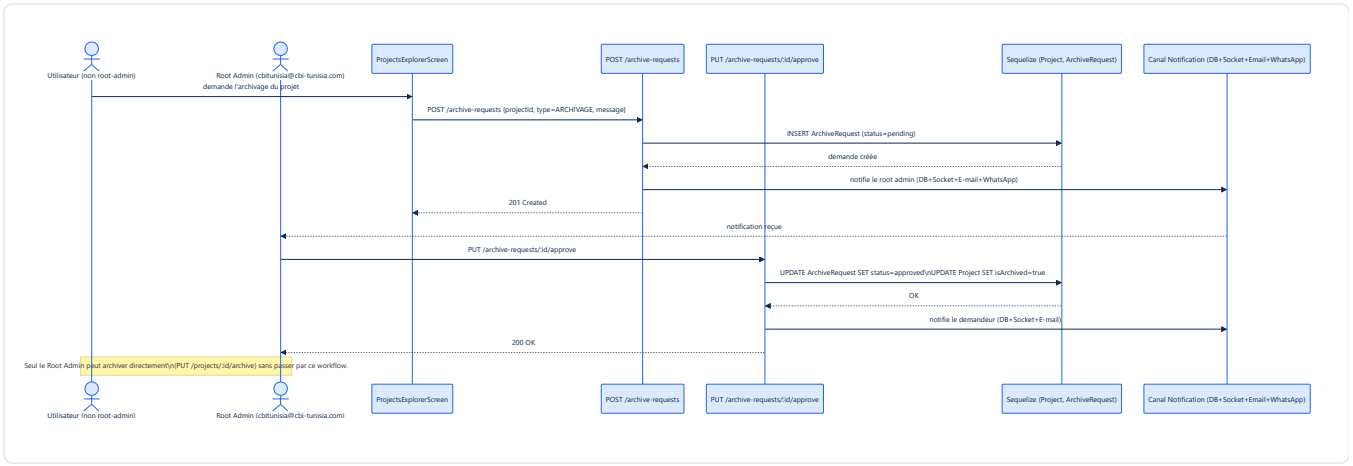


Figure 6.5. « Supprimer un projet » n'existe pas comme suppression physique dans le parcours utilisateur normal : le cycle de vie standard est l'archivage, gouverné par ce workflow d'approbation à deux temps.

6.6 Créer un client (contact commercial)

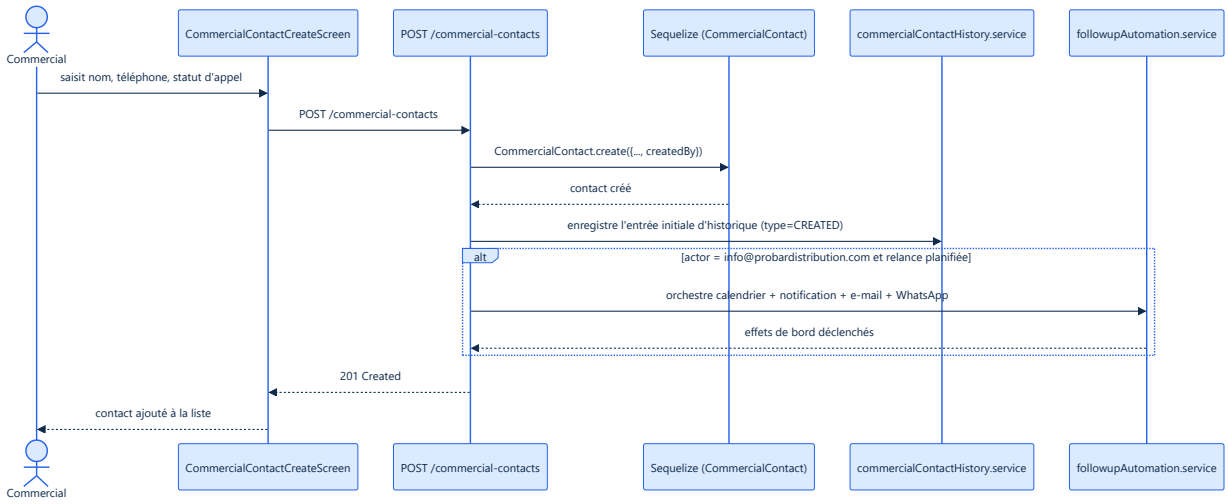
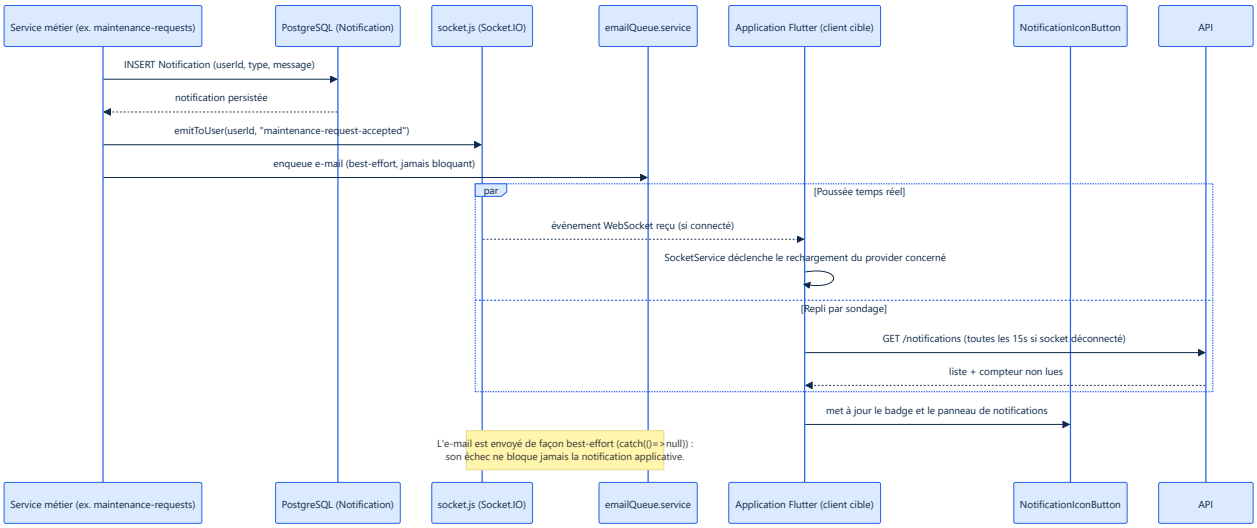


Figure 6.6. Il n'existe pas d'entité `Client` distincte dans le schéma actif : la notion de « client » du CRM correspond fonctionnellement à `CommercialContact` (le module `clients` restant est une table orpheline utilisée uniquement pour l'import CSV/Excel, voir §10.7).

6.7 Notification (temps réel)



6.8 Export PDF (demande RH)

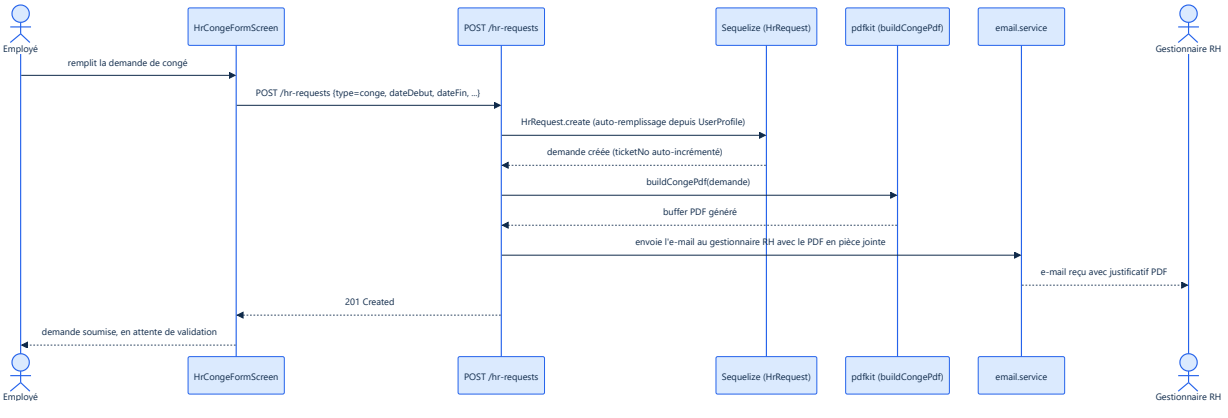
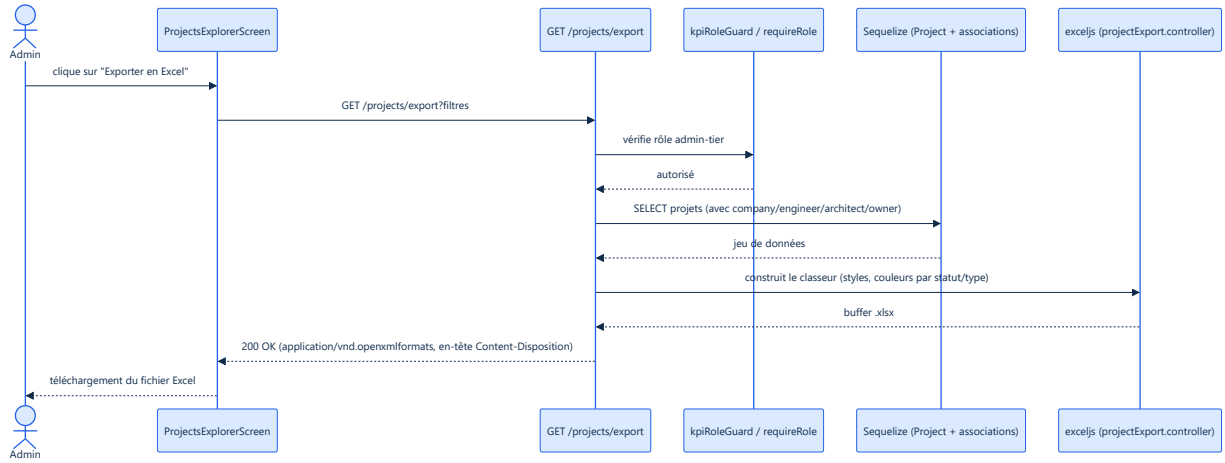
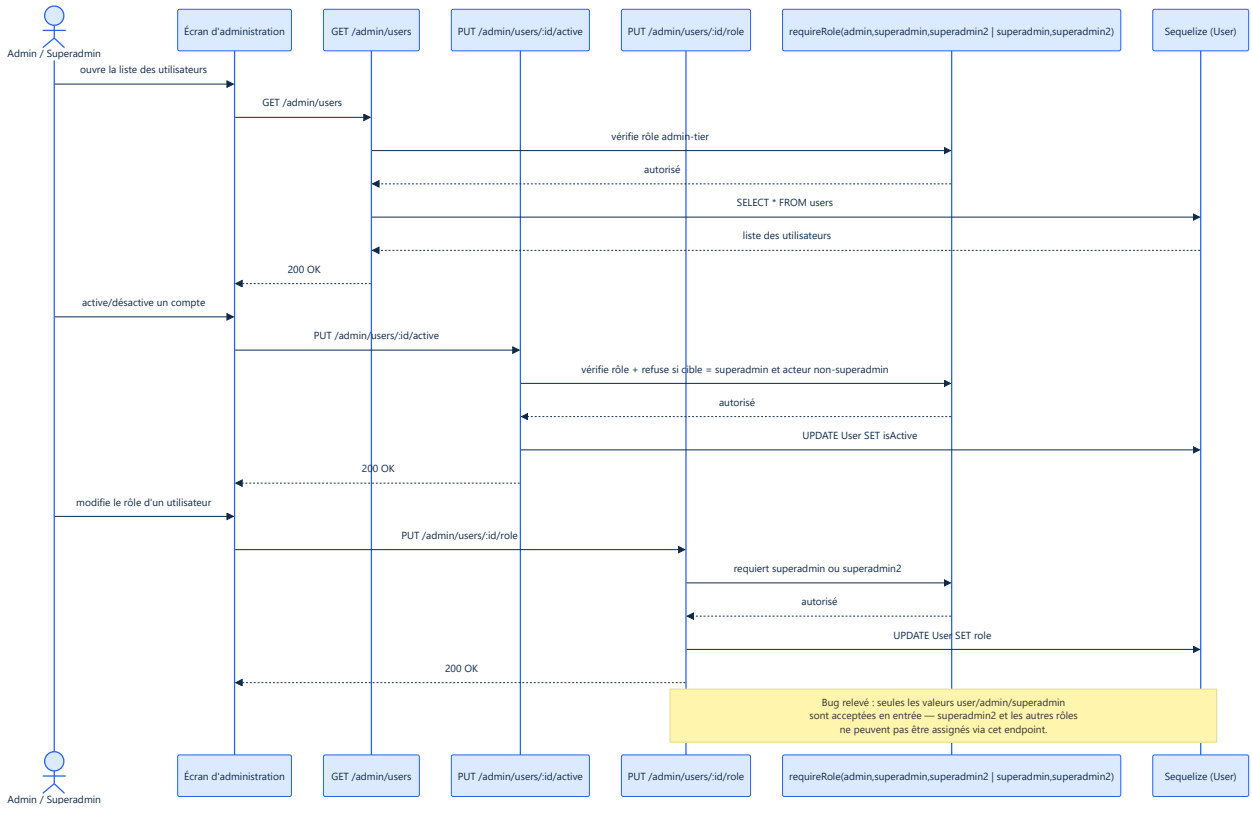


Figure 6.8. Le même mécanisme (`buildSortiePdf`) s'applique aux demandes de sortie ; les rapports POR PROMESH utilisent une génération PDF équivalente via `pdfkit` sur `GET /por-promesh/:id/pdf` .

6.9 Export Excel (liste de projets)

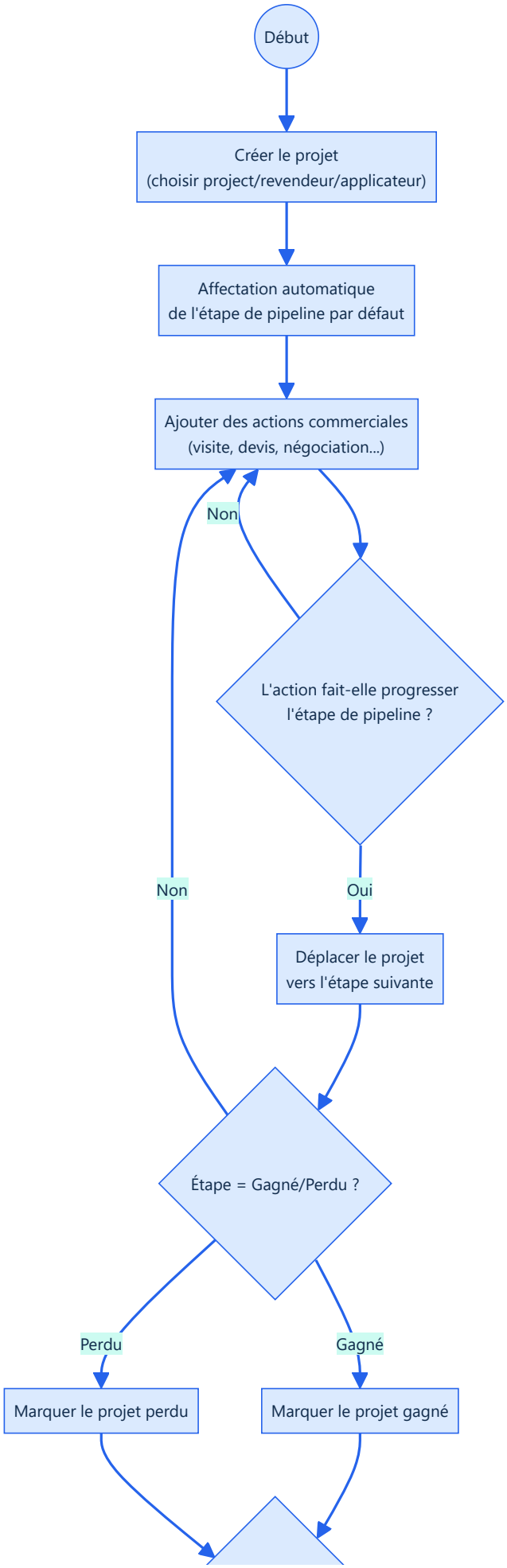


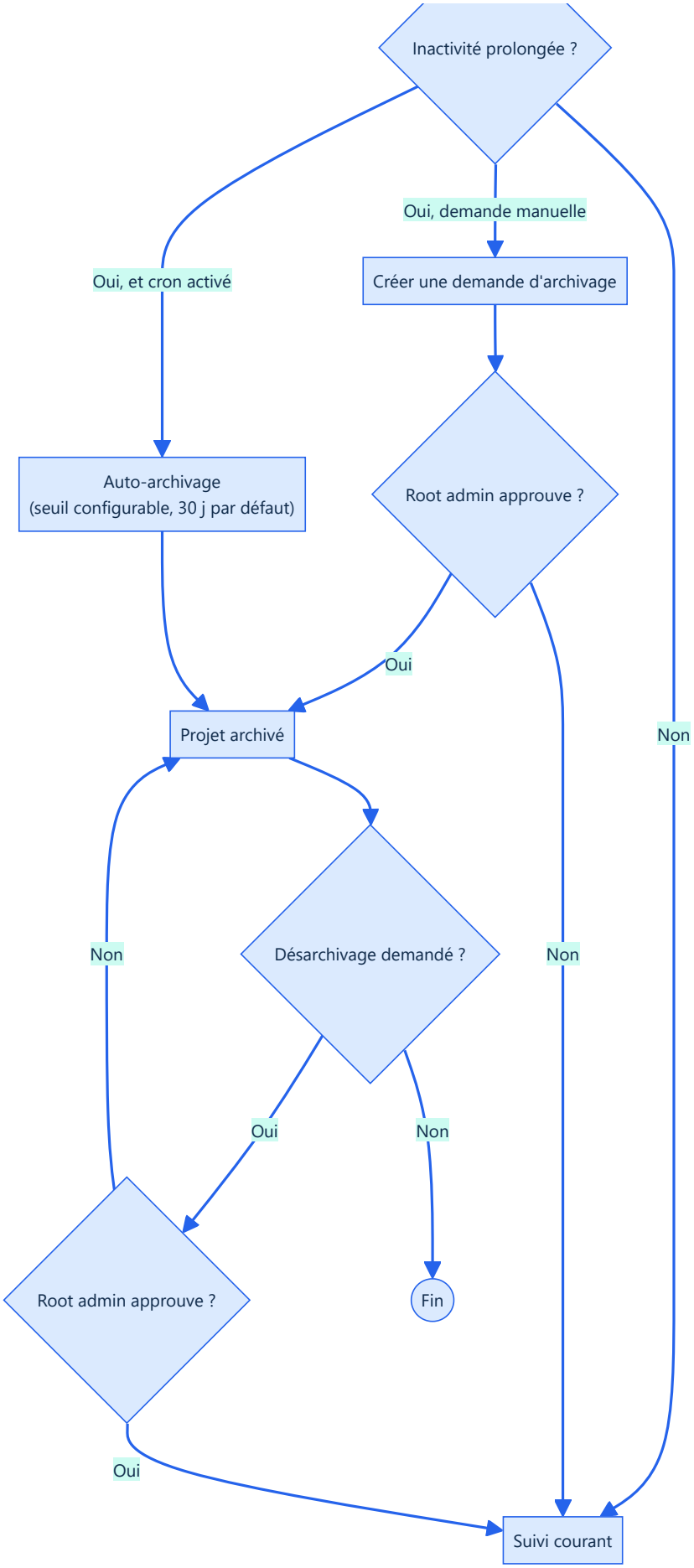
6.10 Gestion des utilisateurs (administration)



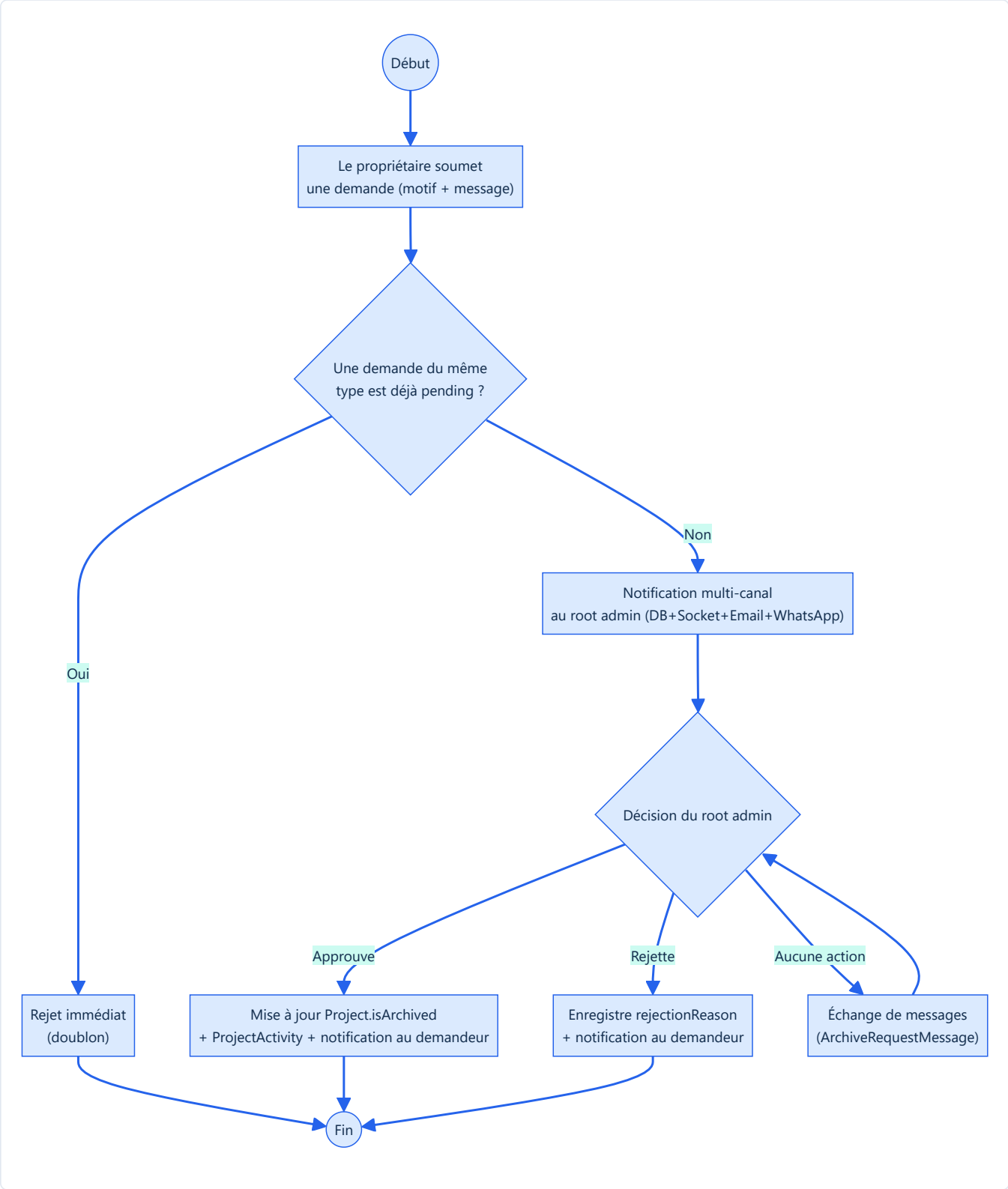
7. Diagrammes d'activité

7.1 Cycle de vie complet d'un projet

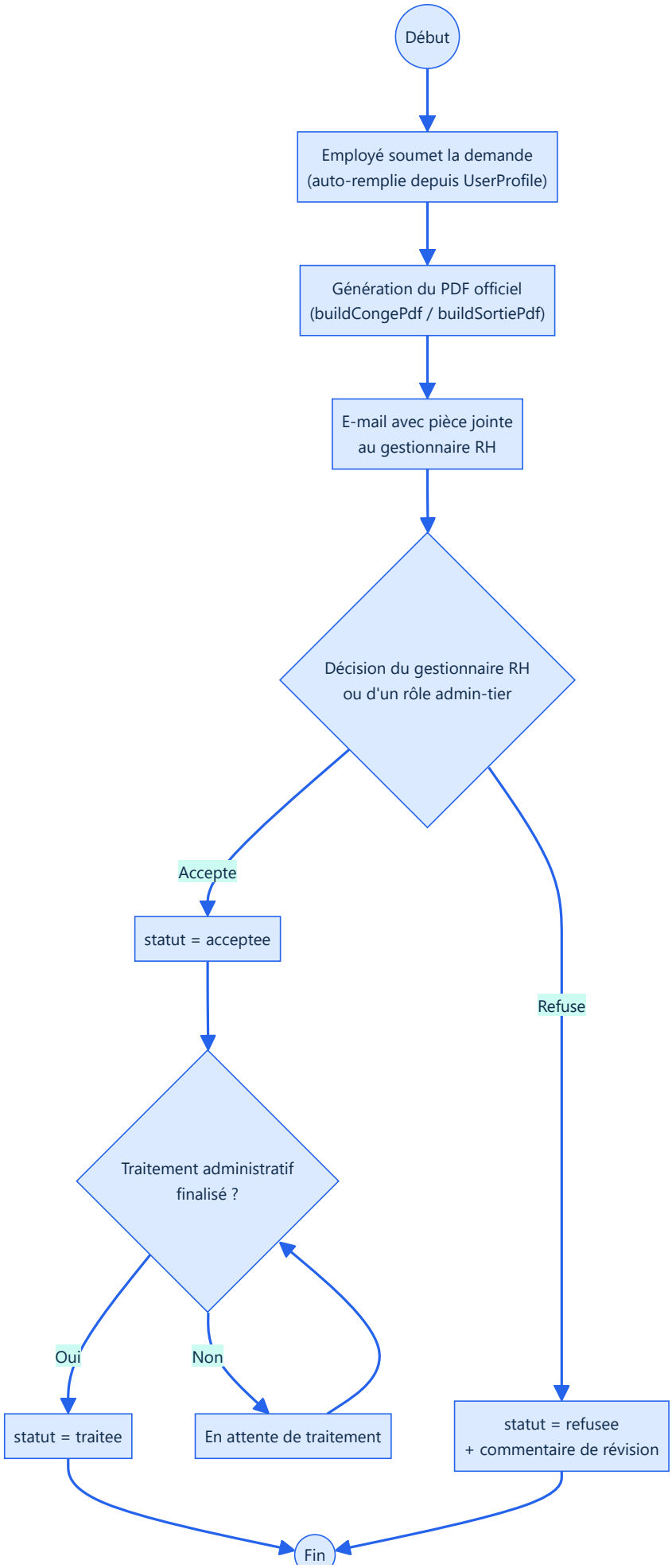

Début
Créer le projet
(choisir project/revendeur/applicateur)
Affectation automatique
de l'étape de pipeline par défaut
Ajouter des actions commerciales
(visite, devis, négociation...)
Non
L'action fait-elle progresser
l'étape de pipeline ?
Non
Oui
Déplacer le projet
vers l'étape suivante
Étape = Gagné/Perdu ?
Perdu
Gagné
Marquer le projet perdu
Marquer le projet gagné


Inactivité prolongée ?
Oui, demande manuelle
Oui, et cron activé
Créer une demande d'archivage
Auto-archivage
(seuil configurable, 30 j par défaut)
Root admin approuve ?
Oui
Non
Projet archivé
Non
Désarchivage demandé ?
Non
Oui
Non
Root admin approuve ?
Fin
Oui
Suivi courant

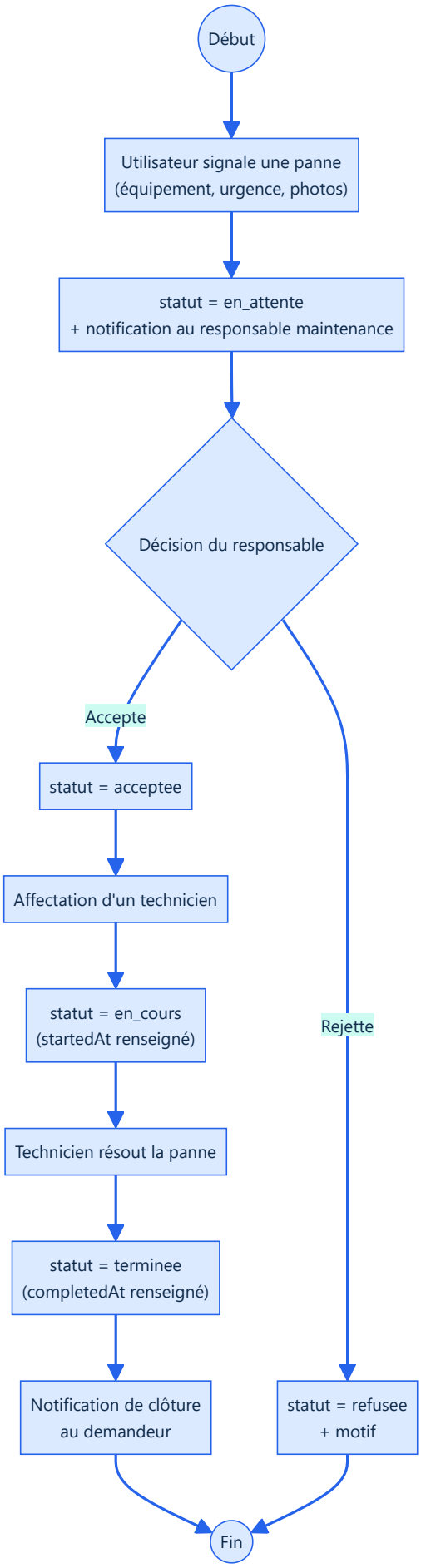
7.2 Demande d'archivage / désarchivage



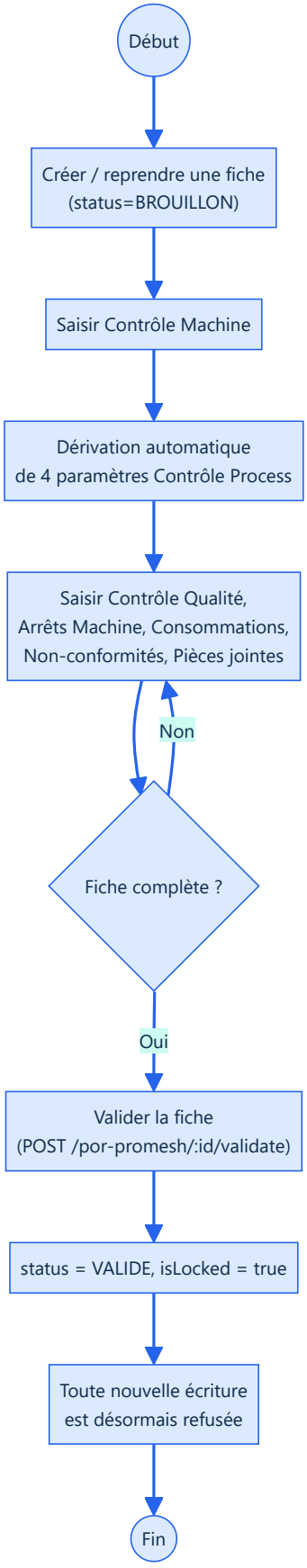
7.3 Demande RH (congé / sortie)


Début
Employé soumet la demande (auto-remplie depuis UserProfile)
Génération du PDF officiel (buildCongePdf / buildSortiePdf)
E-mail avec pièce jointe au gestionnaire RH
Décision du gestionnaire RH ou d'un rôle admin-tier
Accepte
Refuse
statut = acceptee
Traitement administratif finalisé ?
Oui
Non
statut = traitee
En attente de traitement
statut = refusee + commentaire de révision
Fin

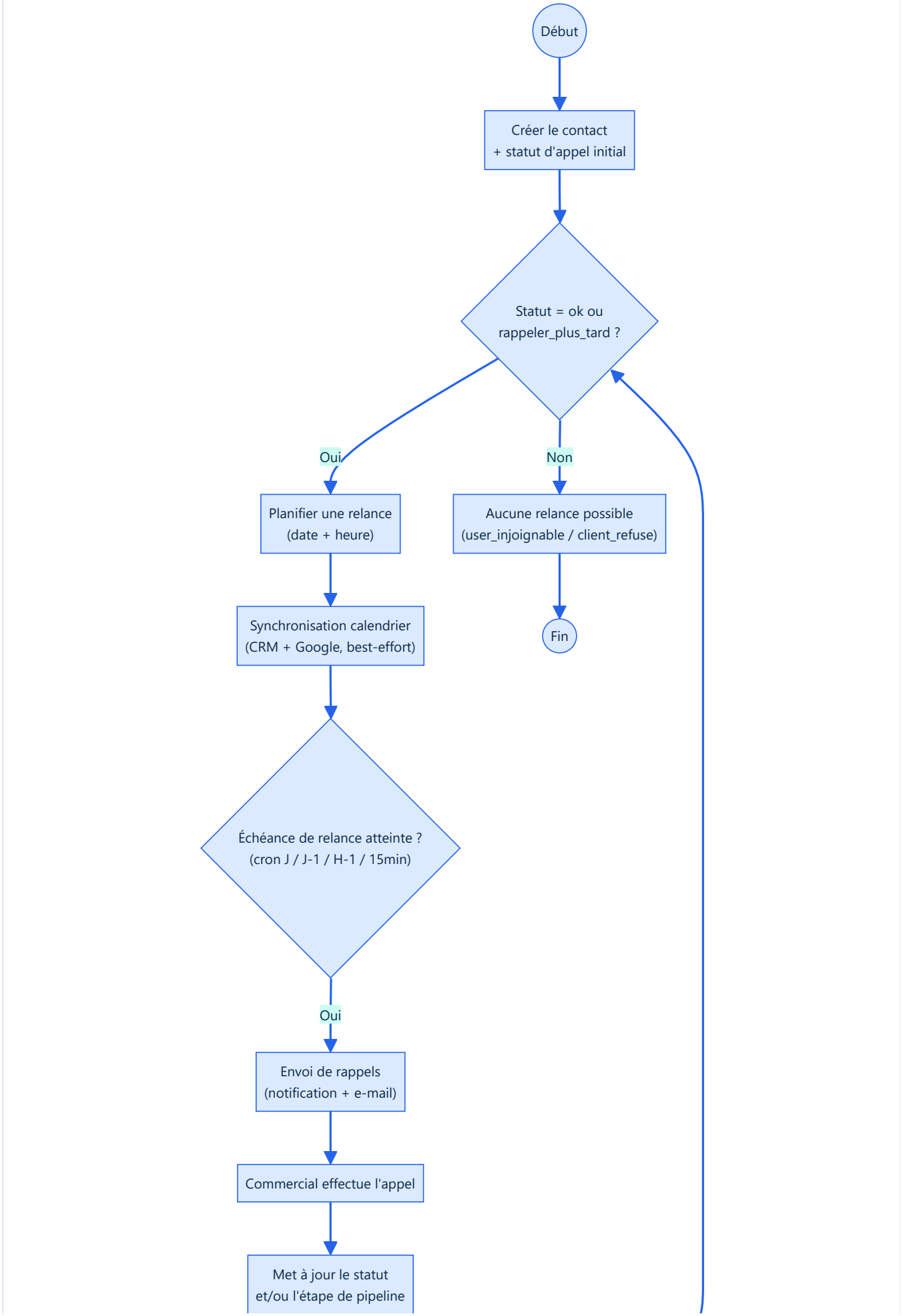
7.4 Demande de maintenance


Début
Utilisateur signale une panne (équipement, urgence, photos)
statut = en_attente + notification au responsable maintenance
Décision du responsable
Accepte
statut = acceptee
Affectation d'un technicien
statut = en_cours (startedAt renseigné)
Technicien résout la panne
statut = terminee (completedAt renseigné)
Notification de clôture au demandeur
Rejette
statut = refusee + motif
Fin

7.5 Saisie et validation d'un rapport POR PROMESH



7.6 Suivi commercial et relance


Début
Créer le contact
+ statut d'appel initial
Statut = ok ou
rappeler_plus_tard ?
Oui
Non
Planifier une relance
(date + heure)
Aucune relance possible
(user_injoignable / client_refuse)
Synchronisation calendrier
(CRM + Google, best-effort)
Fin
Échéance de relance atteinte ?
(cron J / J-1 / H-1 / 15min)
Oui
Envoi de rappels
(notification + e-mail)
Commercial effectue l'appel
Met à jour le statut
et/ou l'étape de pipeline


Écriture dans
CommercialContactStatusHistory

8. Diagrammes d'état

8.1 Projet

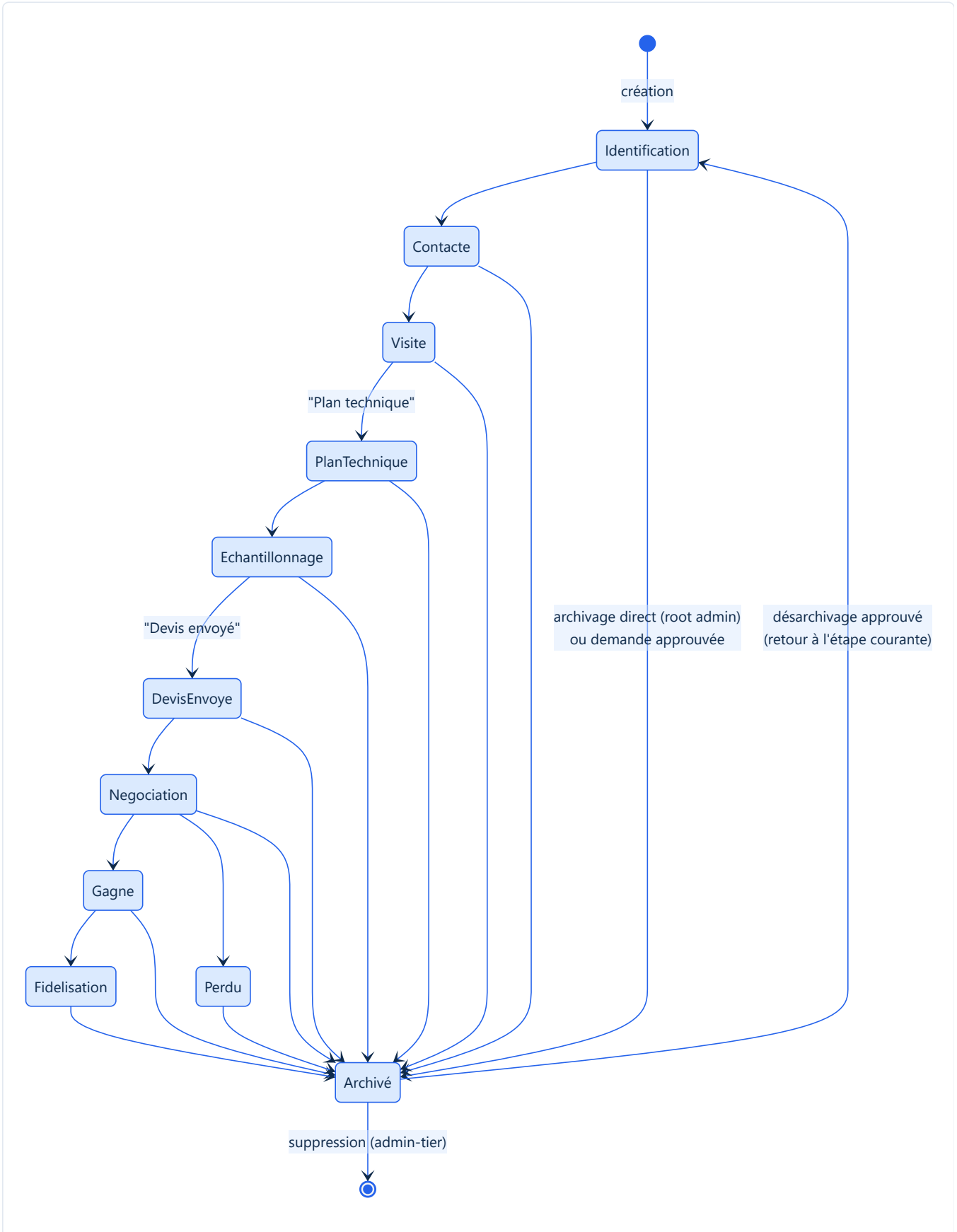


Figure 8.1. Les étapes intermédiaires correspondent aux `PipelineStage` par défaut ; elles sont personnalisables (le pipeline est une donnée, pas un ENUM figé). L'état « Archivé » est orthogonal à l'étape : un projet archivé conserve son étape de pipeline en mémoire pour le désarchivage.

8.2 Client (Contact commercial)

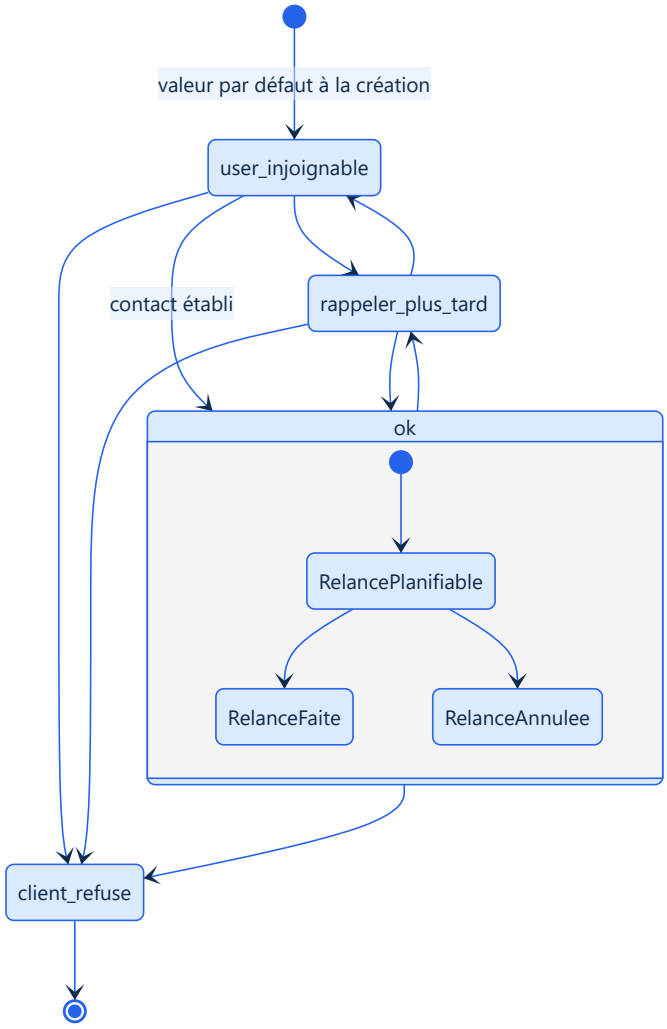


Figure 8.2. Chaque transition de `statut` déclenche une écriture dans `CommercialContactStatusHistory` (non représentée ici car elle n'est pas un état mais un journal). Seuls les statuts `ok` et `rappeler_plus_tard` autorisent la création d'une relance planifiée.

8.3 Document (rapport POR PROMESH)

Aucune entité « Facture » formelle n'existe dans le schéma : le document le plus proche d'un cycle de vie « rédaction → verrouillage définitif » est le rapport POR PROMESH, présenté ici comme représentant de la famille « Document métier ».

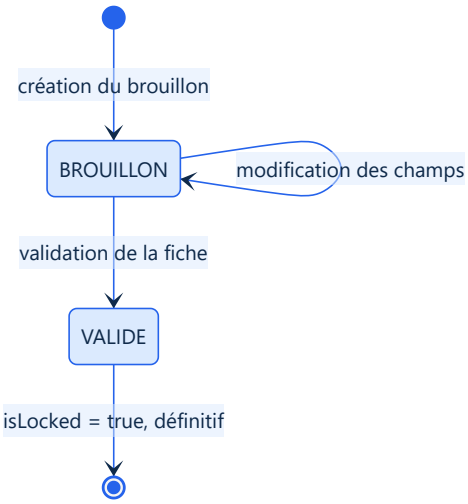
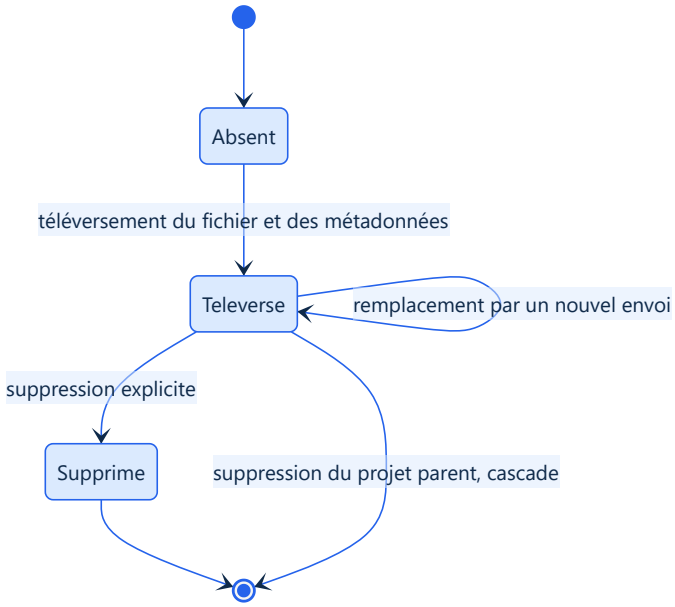


Figure 8.3. Une fois `status=VALIDE` et `isLocked=true`, plus aucune écriture n'est acceptée sur la fiche (irréversible).

8.4 « Facture » — document financier le plus proche (Devis / Bon de commande)

Le schéma ne modélise aucune entité `Facture` / `Invoice` (seule la table orpheline `clients.derniereFacturation` capture une date de facturation, sans workflow). Par analogie fonctionnelle la plus proche dans le cycle commercial, l'état ci-dessous documente le cycle de vie réel d'un **Devis** ou d'un **Bon de commande** attaché à un projet.



8.5 Demande (schéma commun RH / Maintenance / Archivage)

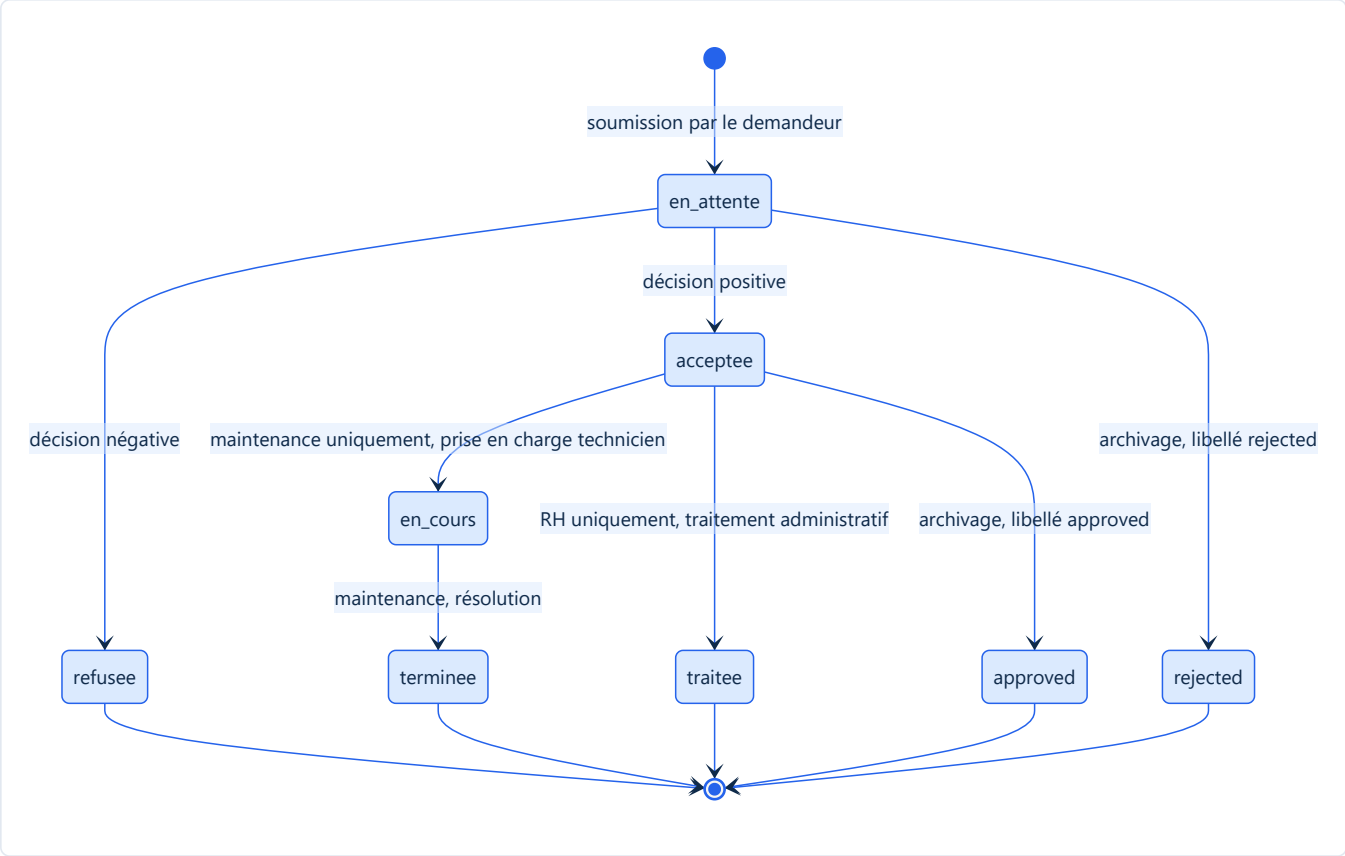
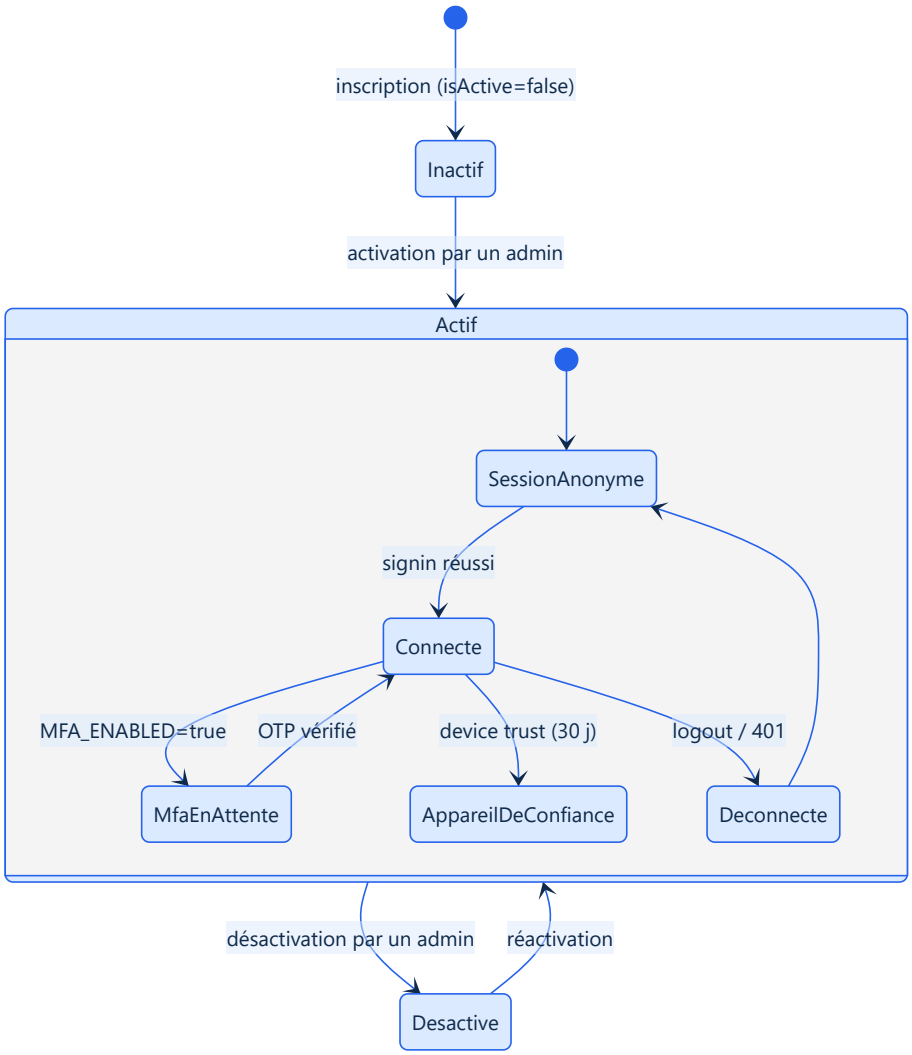


Figure 8.5. Les trois modules de demande (`HrRequest` , `MaintenanceRequest` , `ArchiveRequest`) partagent la même forme d'automate (soumission → décision → clôture) avec un vocabulaire d'états légèrement différent selon le module — un signal DDD fort en faveur d'une généralisation `ApprovalRequest` (voir §5.3 et §14).

8.6 Utilisateur


inscription (isActive=false)
Inactif
activation par un admin
Actif
SessionAnonyme
signin réussi
Connecte
MFA_ENABLED=true
OTP vérifié
device trust (30 j)
logout / 401
MfaEnAttente
AppareilDeConfiance
Deconnecte
désactivation par un admin
réactivation
Desactive

9. Diagramme de communication

Le diagramme de communication met l'accent sur les liens structurels entre objets collaborants plutôt que sur l'ordre temporel (déjà couvert au chapitre 6). Les numéros d'étape correspondent à l'ordre chronologique des échanges.

9.1 Traitement d'une demande de maintenance

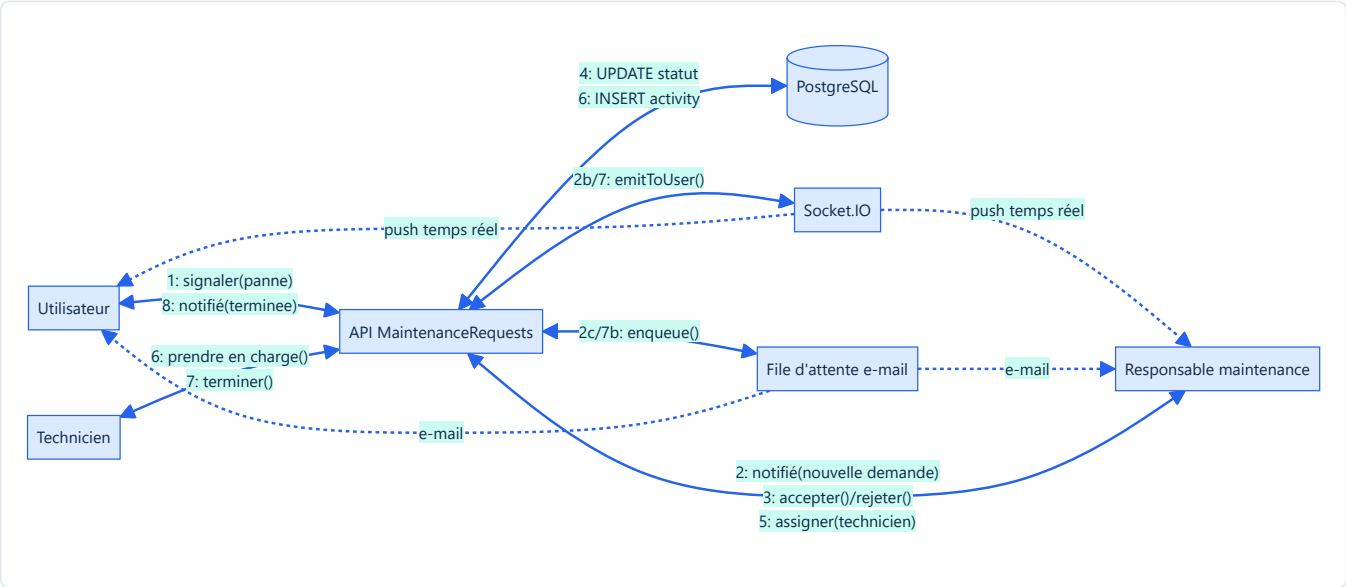


Figure 9.1. Trois collaborateurs (Utilisateur, Responsable maintenance, Technicien) interagissent avec l'objet contrôleur `API MaintenanceRequests`, qui répercute chaque changement d'état vers la base de données et vers les deux canaux de notification (Socket.IO, e-mail).

9.2 Authentification avec garde d'accès par module

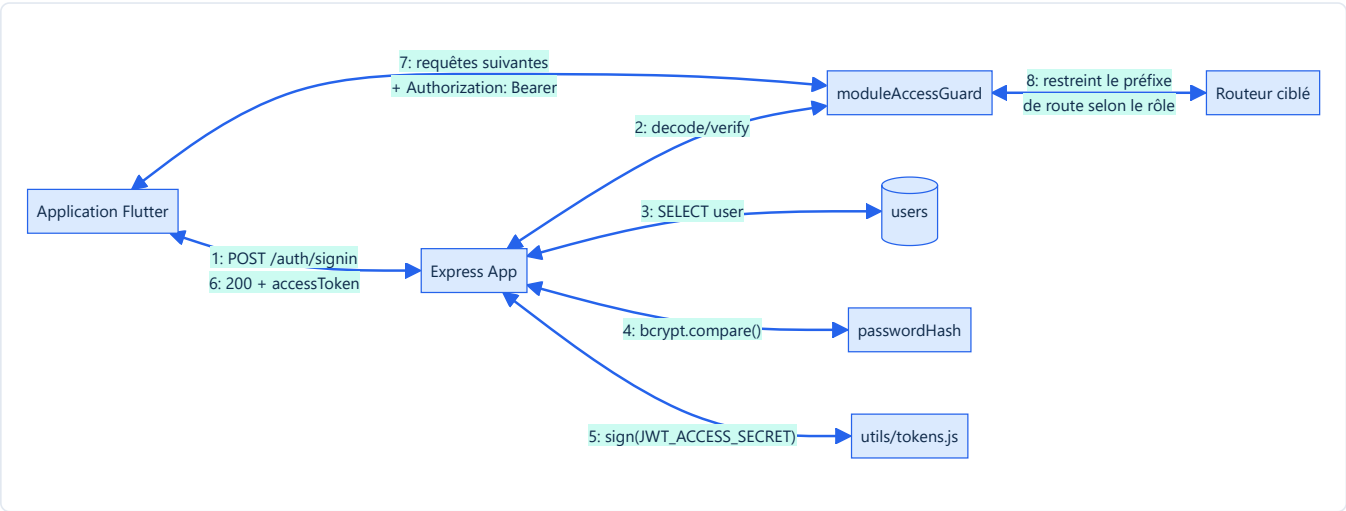


Figure 9.2. `moduleAccessGuard` est un objet de garde global, distinct du middleware `authRequired` utilisé route par route — il réalise son propre décodage du jeton (duplication relevée au §14).

10. Modèle de données (ERD)

Base PostgreSQL, moteur Sequelize, ~50 tables réparties sur 101 fichiers de migration (5 historiques hors `.sequelizerc` + 96 actifs). Aucune vue SQL n'a été trouvée. Pour rester lisible, l'ERD est présenté par grappe fonctionnelle ; les clés étrangères **en gras** dans les tableaux sont physiquement déclarées en base — les autres relations n'existent qu'au niveau applicatif (association Sequelize sans contrainte SQL), une distinction importante détaillée au §10.8.

10.1 Grappe « Identité, sécurité & intégrations »

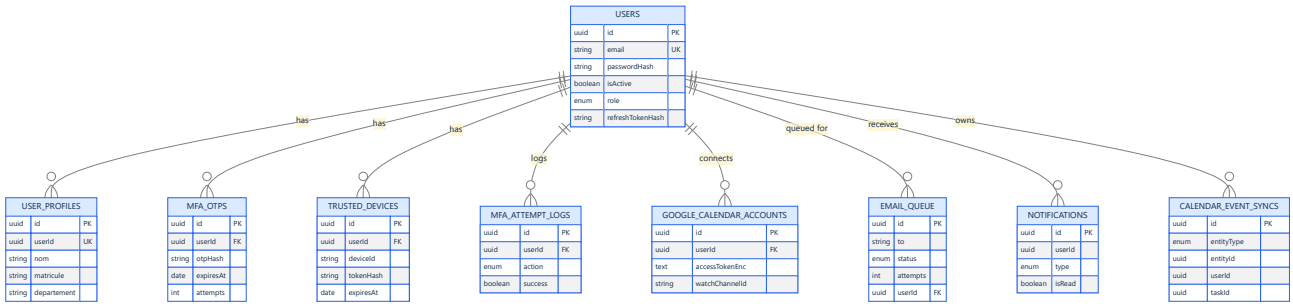


Figure 10.1. `NOTIFICATIONS` porte en réalité 7 pointeurs polymorphes nullables supplémentaires (`projectId, commentId, commercialContactId, relanceId, maintenanceRequestId, hrRequestId, actionId`), aucun n'étant contraint par une clé étrangère — omis ici pour la lisibilité, détaillés au §10.6.

10.2 Grappe « Pipeline & Projets »

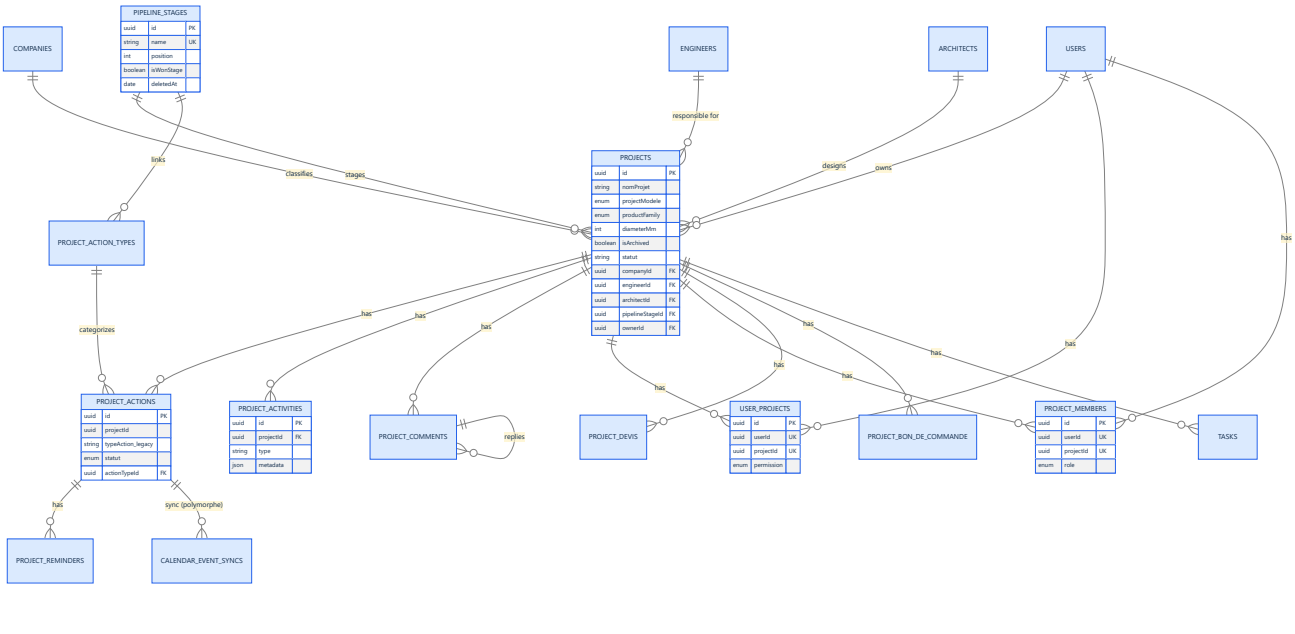


Figure 10.2. `USER_PROJECTS` et `PROJECT_MEMBERS` sont deux tables de jonction M:N parallèles portant la même relation Users↔Projects avec des vocabulaires de permission différents (voir incohérence §10.8-2). Aucune des tables « historiques » (PROJECTS, USERS, PROJECT_ACTIONS…) ne déclare de contrainte FK physique : les liens ci-dessus sont d'origine Sequelize (`associations.js`), pas SQL.

10.3 Grappe « Prospection commerciale »

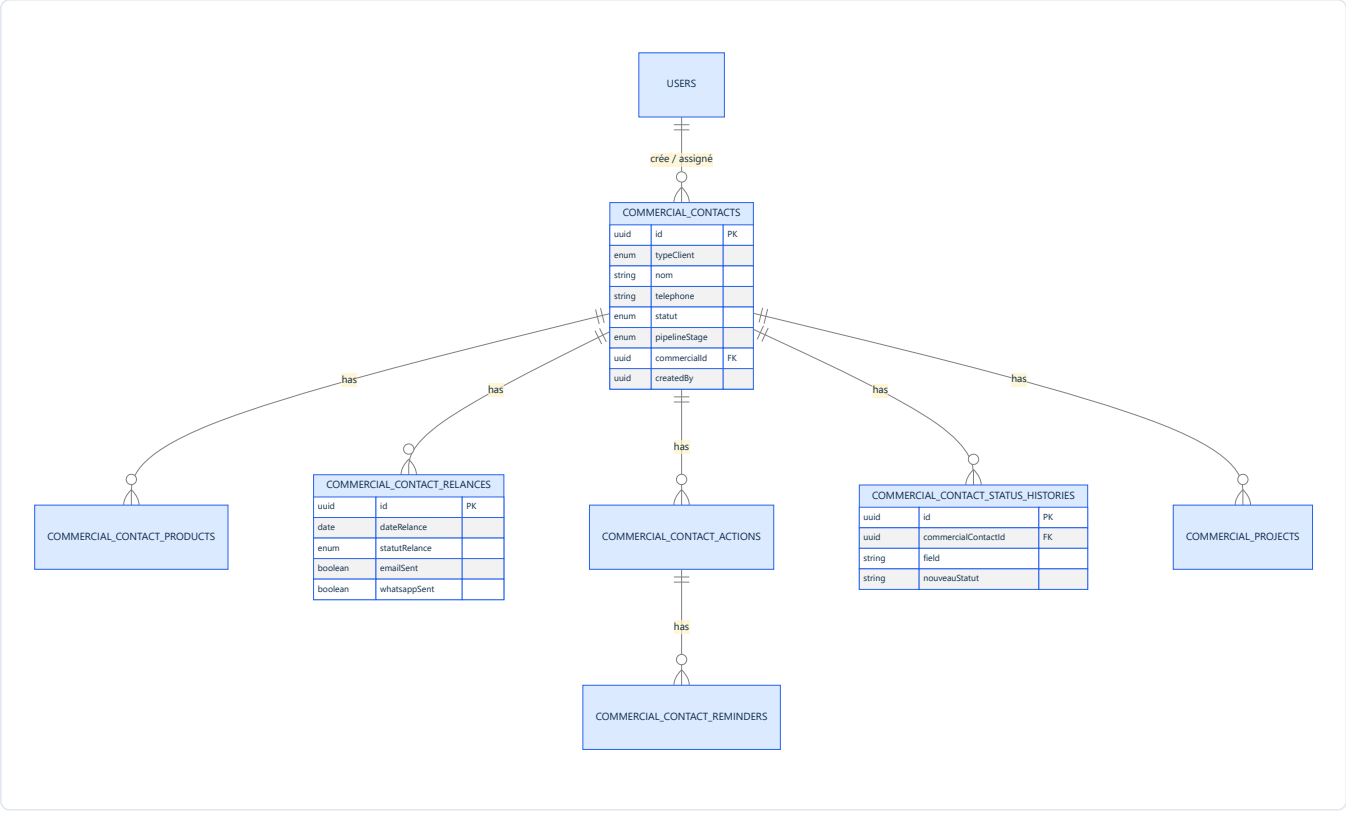


Figure 10.3. Cette grappe duplique presque intégralement la structure Action/Rappel de la grappe Projets (§10.2), en parallèle et sans réutilisation de code — voir §10.8-3/4. `COMMERCIAL_CONTACTS.pipelineStage` reste un ENUM figé à 7 valeurs, alors que `PROJECTS.pipelineStageId` a été migré vers une table dynamique `PIPELINE_STAGES`.

10.4 Grappe « Workflows d'approbation »

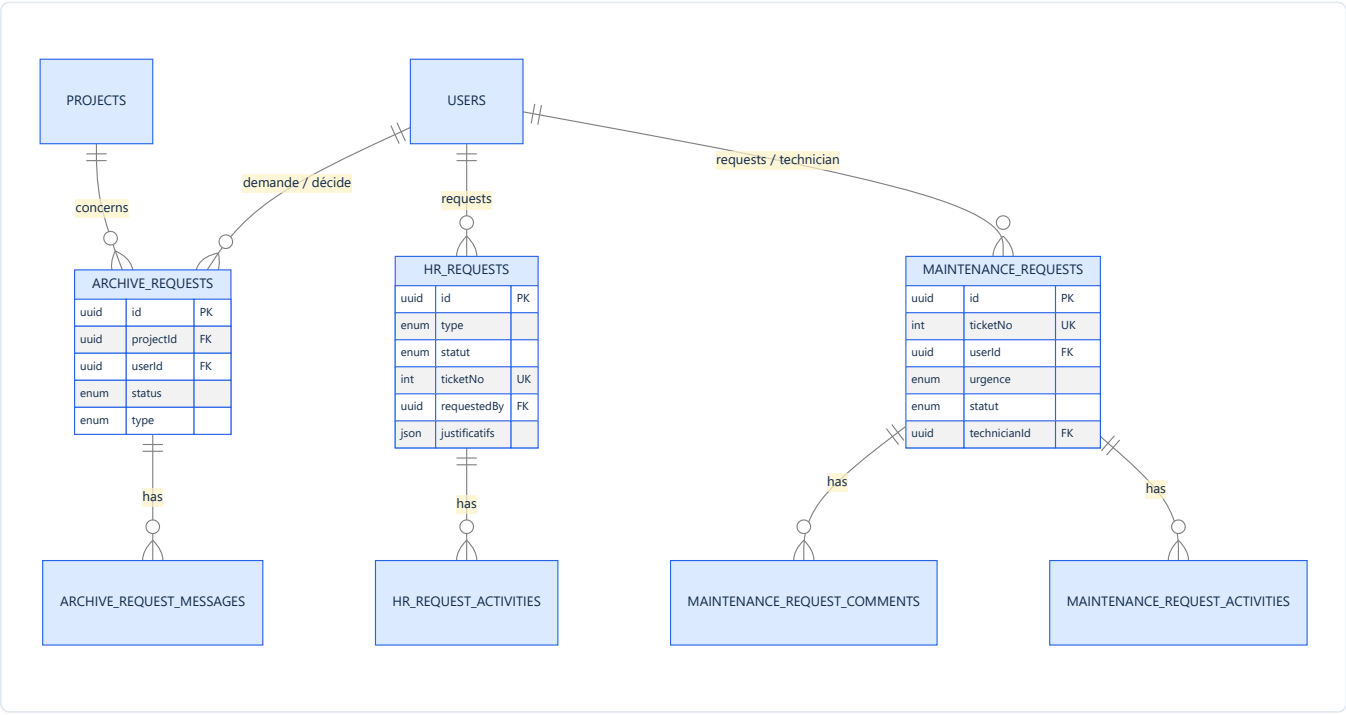


Figure 10.4. Ces trois racines déclarent, contrairement aux tables historiques, de véritables contraintes FK physiques (ajoutées via de vraies migrations) — un contraste net avec le noyau Projets/Users illustrant l'évolution des pratiques dans le temps (voir §10.8-10).

10.5 Grappe « Opérations industrielles »

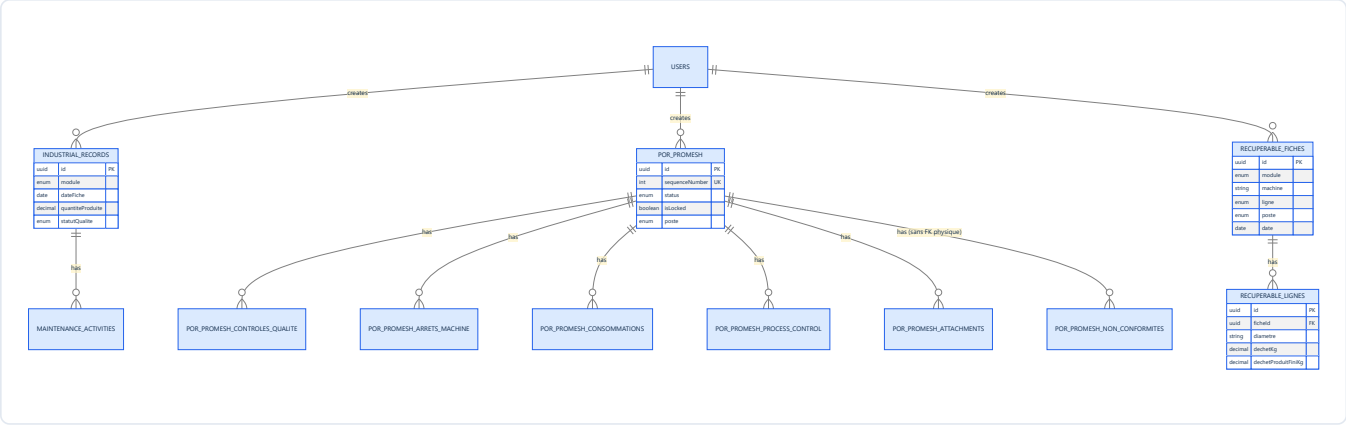


Figure 10.5. `POR_PROMESH_NON_CONFORMITES` est la seule table fille du cluster POR PROMESH sans contrainte FK physique déclarée, contrairement à ses 5 tables sœurs — anomalie relevée au §10.8.

10.6 Inventaire détaillé des tables principales

Table	Colonnes clés	PK	FK physiques	Contraintes notables
users	email, passwordHash, isActive, role (ENUM 7 valeurs), refreshTokenHash, mfaLastVerifiedAt, lastLogin*	id (UUID)	aucune	unique(email) ; ENUM étendu 3× par migration
projects	nomProjet, projectModele, productFamily, diameterMm, isArchived, statut (varchar ex-enum), montantMarche, latitude/longitude, priority	id	companyId→companies (SET NULL), engineerId→engineers, architectId→architects, pipelineStageId→pipeline_stages (SET NULL), ownerId→users (SET NULL)	index trigram GIN (nomProjet, entreprise) via `pg_trgm` ; hook beforeCreate (étape par défaut)
companies / engineers / architects	name (unique), phone, email	id	—	unicité insensible à la casse/espaces (index sur `LOWER(TRIM(name))`)
pipeline_stages	name, color, position, isDefault/isWonStage/isLostStage/autoCreateAction, isCustom	id	—	**paranoid** (deletedAt) — seule table soft-delete de tout le schéma ; unicité partielle du nom (WHERE deletedAt IS NULL)
project_action_types	name (unique), color, linkedStageId	id	linkedStageId→pipeline_stages (SET NULL)	—
project_actions	typeAction_legacy (varchar, ex-ENUM 9 valeurs), actionTypeId, statut, priorite, dateAction, dateRelance, google*	id	actionTypeId→project_action_types (SET NULL), calendarEventId→tasks (SET NULL)	projectId/createdBy sans FK physique ; ex-bug ENUM restrictif corrigé par migration
notifications	type (varchar), title, message, isRead, + 7 pointeurs polymorphes nullables	id	aucune (tous les pointeurs sont non contraints)	aucun contrôle « exactement un pointeur non nul » par type
user_projects	permission (viewer/editor/owner)	id	—	unique(userId, projectId)
project_members	role (viewer/editor)	id	—	unique(projectId, userId) — doublon fonctionnel de user_projects
commercial_contacts	typeClient (ENUM, default invalide « autre »), statut, pipelineStage (ENUM figé), nbAppels, commercialId	id	commercialId→users (SET NULL)	voir anomalie §10.8-6 (default hors domaine ENUM)
commercial_contact_status_histories	field, type, ancienStatut, nouveauStatut, changedBy, changedByName (snapshot)	id	commercialContactId→commercial_contacts (CASCADE)	append-only (pas d'updatedAt)
archive_requests	status (pending/approved/rejected), type (ARCHIVAGE/DESARCHIVAGE), rejectionReason	id	projectId→projects (CASCADE), userId/adminId/approvedBy/rejectedBy→users	—
maintenance_requests	ticketNo (auto-increment, unique), equipement, urgence, statut, photos (JSONB)	id	userId, technicianId, assignedBy, startedBy, acceptedBy, rejectedBy, completedBy → users (SET NULL sauf userId CASCADE)	—

hr_requests	type (conge/sortie), statut, ticketNo (auto-increment, unique), employeeNom/Prenom/... (snapshot), justificatifs (JSONB)	id	requestedBy→users (CASCADE), reviewedBy/processedBy→users (SET NULL)	—
industrial_records	module (probar/melange/maintenance), dateFiche, quantiteProduite, statutQualite, melangeData (JSONB)	id	createdBy→users (CASCADE)	index composite (module, dateFiche, createdAt)
por_promesh	sequenceNumber (séquence Postgres dédiée), status (BROUILLON/VALIDE), isLocked, poste, conformite	id	createdBy→users (CASCADE)	plusieurs colonnes abandonnées au fil des migrations (longueur, largeur, bainResine...)
por_promesh_non_conformites	typeNC, gravite, description, photoUrl	id	**aucune** (contrairement à ses 5 tables sœurs)	anomalie — voir §10.8
recuperable_fiches	module (PROBAR/PROMESH), machine, ligne (L1-L4), poste, statut (en_cours/cloturee)	id	createdBy→users (CASCADE)	unique(module, machine, ligne, poste, date)
email_queue	to, subject, status (PENDING/RETRYING/SENT/FAILED/CANCELLED), attempts, maxAttempts	id	userId→users (SET NULL)	reprise automatique au démarrage du serveur
mfa_otps / trusted_devices / mfa_attempt_logs	otpHash/tokenHash (jamais en clair), expiresAt, attempts	id	userId→users (CASCADE / SET NULL)	index (userId, deviceId) **non unique** sur trusted_devices — anomalie
clients (table orpheline)	code, raisonSociale, matriculeFiscal, derniereFacturation	id (**INTEGER auto-incrément**, seule table hors UUID)	aucune, non déclarée dans associations.js/models/index.js	utilisée uniquement par `client.routes.js` (import CSV/Excel) — voir §10.8-7

10.7 Tables de jonction & données de référence

Table de jonction	Relie	Attribut porté
user_projects	users ↔ projects	permission (viewer/editor/owner) — source de vérité historique de `projects.ownerId`
project_members	users ↔ projects	role (viewer/editor) — doublon fonctionnel
calendar_event_syncs	(project_action | commercial_contact_relance) ↔ users	métadonnées de synchronisation Google (polymorphe par `entityType`)

Données de référence (seeders) : `pipeline_stages` — 10 étapes seedées (Prospect, Contacté, Site Visit, Plan technique, Échantillonnage, Quote Sent, Negotiation, Won, Lost, Loyalty) ; `project_action_types` — 5 types « core » + 9 types « legacy » + auto-générés par étape ; comptes de référence `cbitunisia@cbi-tunisia.com` (superadmin), `responsable_logistique@cbi-tunisia.com`, `manegerofficecbi@gmail.com`.

10.8 Constats structurels de la base de données

Écart logique/physique : toutes les tables « historiques » (users, projects, notifications, tasks, project_actions, project_comments, user_projects, project_members, commercial_contacts et satellites, project_reminders) ne déclarent **aucune contrainte FK physique** : les relations n'existent qu'au niveau Sequelize (`associations.js`). À l'inverse, chaque table ajoutée par une vraie migration (archive_requests, maintenance_requests, hr_requests, por_promesh*, mfa_*, email_queue) déclare de vraies références SQL. L'ERD « logique » est donc plus riche que le schéma réellement contraint par PostgreSQL.

Duplication de jonction M:N : `user_projects` et `project_members` relient les deux mêmes entités avec deux vocabulaires de permission différents — candidat à une consolidation.

Duplication de concept « action »/« rappel » : `project_actions` + `project_reminders` et `commercial_contact_actions` + `commercial_contact_reminders` sont des schémas quasi identiques maintenus indépendamment pour deux parents différents.

Deux représentations du entonnoir de vente : `commercial_contacts.pipelineStage` reste un ENUM figé à 7 valeurs alors que `projects.pipelineStageId` a été migré vers la table dynamique `pipeline_stages`.

Valeur par défaut hors domaine : `commercial_contacts.typeClient` est un ENUM(Tuteur, Cloture, Batiment) avec `defaultValue:'autre'`, qui n'appartient pas à l'ENUM — une insertion sans valeur explicite échouerait.

Table orpheline : `clients` n'est référencée dans aucune association Sequelize, utilise une clé primaire entière auto-incrémentée (seule exception à l'usage systématique d'UUID) et reste pourtant exposée par une route active (`client.routes.js`).

FK manquante isolée : `por_promesh_non_conformites.porPromeshId` n'a pas de contrainte FK physique contrairement à ses 5 tables sœurs — une suppression brute en base pourrait orpheliner ces lignes malgré la cascade déclarée côté Sequelize.

Index non-unique inattendu : `trusted_devices(userId, deviceId)` n'est pas contraint en unicité malgré l'attente naturelle d'une seule ligne active par appareil et par utilisateur.

Bug historique corrigé : `project_actions.typeAction_legacy` a longtemps été un ENUM Postgres à 9 valeurs restrictif malgré un modèle Sequelize déclaré en `STRING(100)`, provoquant des erreurs HTTP 500 pour tout type hors liste jusqu'à la migration correctrice.

Vocabulaires de priorité divergents : `projects.priority` est en anglais libre (low/medium/high/urgent) tandis que `project_actions.priorite` / `tasks.priority` utilisent un ENUM français (basse/normale/haute/urgente) — même concept, deux vocabulaires.

11. Description module par module

Chaque module ci-dessous est décrit selon la même grille de lecture : objectif métier, fonctionnalités clés, règles de gestion effectivement codées, dépendances vers d'autres modules, écrans Flutter branchés, et permissions. Les chemins de fichiers backend sont donnés sous `src/` (racine `Backend Master`).

11.1 Pipeline

Objectif	Définir et administrer les étapes du entonnoir de vente utilisées par Projects et Kanban.
Fonctionnalités	CRUD des étapes, réordonnancement (`PUT /reorder`), marquage « étape gagnée/perdue/par défaut », auto-création de type d'action lié.
Règles métier	Nom d'étape unique (parmi les non supprimées) ; suppression bloquée (409) si des projets utilisent l'étape ; soft-delete (paranoid).
Dépendances	Consommé par Projects, Kanban ; produit des `ProjectActionType` .
Permissions	Lecture : tout utilisateur authentifié. Écriture : admin-tier uniquement (`requireRole`).
Fichiers clés	`modules/pipeline/{routes,controllers,services,repositories}`

11.2 Projects

Objectif	Entité centrale du CRM : chantiers, prospects revendeurs/applicateurs.
Fonctionnalités	CRUD complet, filtrage/recherche (trigram), export Excel, KPI dédiés, changement d'étape/propriétaire/statut, archivage direct (root admin) ou via workflow, timeline, notes.
Règles métier	Champs obligatoires dépendant de `projectModele` ; cohérence `productFamily` / `diameterMm` ; étape par défaut auto-assignée à la création ; deux routeurs (legacy + module) cohabitent sur le même préfixe, désambiguïsés par un test de format UUID.
Dépendances	Pipeline (étape), Companies/Engineers/Architects (référentiels), Project-Actions, Project-Activities, Kanban, Archive-Requests, Users (propriétaire).
Écrans Flutter	`ProjectFormScreen` , `ProjectsExplorerScreen` , `ApplicateurProjectsScreen` , `RevendeurProjectsScreen` , `MissingFieldsProjectsScreen` , `ProjectPipelineBoard` .
Permissions	Propriétaire ou admin-tier pour l'écriture ; archivage direct réservé au root admin.

11.3 Project-Actions & Project-Activities

Objectif	Project-Actions : journal d'activités commerciales exploitables (visites, devis, négociations). Project-Activities : fil d'audit technique en lecture seule.
Fonctionnalités	CRUD d'actions avec pièce jointe, rappels, synchronisation Google/CRM Calendar (best-effort), CRUD des types d'action ; lecture de la timeline d'activités.
Règles métier	Chaque action crée un rappel et met à jour `Project.lastRelanceAt` ; `ProjectActivity` n'est jamais modifiée/supprimée, uniquement ajoutée par les autres services.
Dépendances	Projects (parent), Pipeline (types liés), `googleCalendar.service` , `Tasks` (agenda CRM).
Permissions	Propriétaire du projet ou admin-tier.

11.4 Kanban

Objectif	Vue tableau glisser-déposer sur les projets, alignée sur les étapes de pipeline.
Fonctionnalités	`GET /kanban` (données groupées par étape), `PUT /projects/:id/stage` (déplacement rapide, sans journal d'activité).
Règles métier	Version allégée du changement d'étape (pas d'écriture d'activité), optimisée pour la fluidité du glisser-déposer.
Écrans Flutter	`KanbanViewScreen` (actuellement branché sur des données de démonstration côté template, à distinguer de `ProjectPipelineBoard` qui, lui, consomme l'API réelle).
Permissions	Tout utilisateur authentifié ayant accès au projet.

11.5 Commercial-Contacts

Objectif	Gestion des prospects/leads en amont du cycle « Projects », avec suivi d'appels et relances.
Fonctionnalités	CRUD contact, assignation en masse à un commercial (admin), relances planifiées, actions, historique de statut, KPI/analytics par commercial.
Règles métier	Portée « commercial » limitée aux contacts créés/assignés/avec relance ; relance possible uniquement si statut ∈{ok, rappeler_plus_tard} ; historique de statut en écriture seule ; automatisation spéciale pour l'acteur `info@probardistribution.com` .
Dépendances	`followupAutomation.service` , `multiRecipientCalendarSync.service` , `whatsapp.service` , `emailQueue.service` .
Écrans Flutter	`CommercialContactListGetxScreen` , `CommercialContactCreateScreen` , `CommercialTimelineScreen` , `AddCommercialActionScreen` , `CommercialContactsAnalyticsScreen` , `CommercialSelectionScreen` .
Permissions	Commercial (portée limitée), admin-tier (accès total), KPI globaux réservés admin-tier + commercial (les siens).

11.6 Archive-Requests

Objectif	Gouvernance de l'archivage/désarchivage des projets par un circuit d'approbation.
Fonctionnalités	Soumission par le propriétaire, messagerie associée à la demande, approbation/rejet par le root admin, statistiques.
Règles métier	Un seul type de demande pending à la fois par projet ; seul `cbitunisia@cbi-tunisia.com` peut décider (ou archiver directement en contournant le workflow).
Événements	Notification DB + Socket.IO + e-mail + WhatsApp (Twilio, si configuré) à chaque étape.
Écrans Flutter	`ArchiveRequestsPage` , `ArchiveRequestChatScreen` , `ArchiveRequestDialog` .
Permissions	Création : tout utilisateur (propriétaire du projet). Décision : root admin uniquement.

11.7 Maintenance-Requests

Objectif	Signalement et suivi des pannes d'équipement jusqu'à résolution.
Fonctionnalités	Création avec photos, acceptation/rejet, affectation technicien, cycle de vie complet, commentaires, historique d'activité, statistiques.
Règles métier	Séquence stricte `en_attente→acceptee→en_cours→terminee` (ou `refusee`) ; distinct du sous-module « maintenance » d'Industrial-Records (précisé explicitement dans le code).
Écrans Flutter	`MaintenanceRequestsPage` , `MaintenanceRequestDetailScreen` , `MaintenanceRequestDialog` — temps réel via `SocketService` + repli par sondage 15s.
Permissions	Création : tout utilisateur. Instruction : admin-tier, `responsable_logistique_achat` , ou `accueilcbif@gmail.com` .

11.8 HR-Requests

Objectif	Demandes de congé et d'autorisation de sortie des employés.
Fonctionnalités	Soumission auto-remplie depuis le profil, génération PDF officiel, e-mail au gestionnaire RH avec pièce jointe, historique, décision.
Règles métier	Décompte des jours en jours ouvrés (lun-ven, sans calendrier de jours fériés) ; `traitee` uniquement accessible depuis `acceptee` ; identité employé figée en copie (snapshot) au moment de la demande.
Écrans Flutter	`HrTypeScreen` , `HrCongeFormScreen` , `HrSortieFormScreen` , `HrHistoryScreen` , `HrDetailScreen` , `HrRequestsAdminPage` .
Permissions	Création : tout utilisateur. Instruction : admin-tier ou `manegerofficecbi@gmail.com` .

11.9 Industrial-Records

Objectif	Fiches de production journalières pour 3 sous-domaines (PROBAR, MÉLANGE, Maintenance industrielle).
Fonctionnalités	CRUD par poste (matin/nuit), historique, champ libre `melangeData` (JSONB) pour la variante mélange.
Règles métier	Portée par propriétaire pour `responsable_logistique_achat` ; création d'un enregistrement `module=maintenance` réservée à l'adresse `responsable_logistique@cbi-tunisia.com` , notifiant `accueilcbif@gmail.com` via un circuit e-mail dédié indépendant du module Maintenance-Requests.
Écrans Flutter	`MachineOverviewScreen` , `ProbarFormScreen` , `MelangeFormScreen` / `MelangeHistoryScreen` , `IndustrialHistoriqueScreen` , `MaintenanceFormScreen` .
Permissions	Écriture : admin-tier + `responsable_logistique_achat` (portée limitée). Suppression : admin-tier uniquement.

11.10 Por-Promesh

Objectif	Rapport d'équipe détaillé pour la ligne de production PROMESH.
Fonctionnalités	Brouillon éditable multi-sections (Contrôle Machine, Contrôle Qualité, Arrêts, Consommations, Contrôle Process, Non-conformités, Pièces jointes), validation définitive, export PDF, tableau de bord/statistiques.
Règles métier	Verrouillage irréversible après validation ; dérivation automatique de 4 paramètres « Contrôle Process » à partir des saisies « Contrôle Machine » à chaque sauvegarde ; numérotation via séquence PostgreSQL dédiée.
Écrans Flutter	`PorPromeshDashboardScreen` , `PorPromeshListScreen` , `PorPromeshDetailScreen` , `PorPromeshHistoriqueScreen` , `PorPromeshStatistiquesScreen` + services d'export PDF/Excel dédiés.
Permissions	Portée par propriétaire pour `responsable_logistique_achat` ; admin-tier voit tout.

11.11 Recuperables

Objectif	Suivi des déchets/matières récupérables par machine, ligne, poste et date.
Fonctionnalités	Upsert par combinaison unique (module+machine+ligne+poste+date), lignes par diamètre, clôture manuelle ou automatique.
Règles métier	Fermeture automatique après 6 jours (`OPEN_WINDOW_DAYS`) si non clôturée manuellement.

Écrans Flutter	`RecuperableFicheScreen` , `RecuperableHistoryScreen` , `RecuperableDetailScreen` , `RecuperableStatsScreen` .
Permissions	Identique au schéma Industrial-Records/Por-Promesh (portée propriétaire pour `responsable_logistique_achat`).

11.12 Dashboard & Admin-Dashboard

Objectif	Agrégations et KPI transverses, en lecture seule, sur l'ensemble des modules métier.
Fonctionnalités	KPI personnels (`/dashboard/user`), KPI globaux et « professional » (`kpiRoleGuard`), statistiques consolidées cross-module (`/admin-dashboard/stats` : PROMESH, PROBAR, Maintenance, RH, Mélange, Récupérables).
Dépendances	Requêtes SQL brutes contre `projects` , `users` , `pipeline_stages` , `industrial_records` , `por_promesh` , `hr_requests` , `recuperable_fiches/lignes` .
Écrans Flutter	`DashboardScreen` (académique/sales/finance/ecommerce — même écran, gabarit), `IndustrialSuperAdminDashboardScreen` (réel), `CommercialContactsKpiScreen` .
Permissions	KPI globaux : admin-tier uniquement. KPI personnels : tout utilisateur (ses propres données).

12. Référentiel d'API

Inventaire des points d'entrée REST exposés par `Backend Master`, regroupés par préfixe de montage. Sauf indication contraire, chaque route passe d'abord par `moduleAccessGuard` (garde globale) puis par `authRequired` (vérification JWT).

12.1 `/auth` — Authentification

Méthode	Route	Protection	Description
POST	/forgot-password	Limiteur 10/h/IP	Demande de réinitialisation (réponse neutre systématique)
GET	/reset-password/validate	Public	Vérifie la validité d'un jeton de réinitialisation
POST	/reset-password	Limiteur 20/h/IP	Définit le nouveau mot de passe
POST	/signup	Public	Création de compte (inactif par défaut, rôle auto selon domaine e-mail)
POST	/signin	Limiteur 20/10min/IP	Connexion, émission des jetons
POST	/mfa/verify	MFA activée + limiteur	Validation du code OTP
POST	/mfa/resend	MFA activée + limiteur	Renvoi du code OTP (cooldown 60s)
POST	/refresh	Cookie refresh	Rotation du jeton d'accès
POST	/logout	—	Invalide le jeton de rafraîchissement

12.2 `/admin` — Administration

Méthode	Route	Rôles	Description
GET	/users	admin-tier	Liste tous les utilisateurs
PUT	/users/:id/active	admin-tier	Active/désactive un compte
PUT	/users/:id/role	superadmin, superadmin2	Change le rôle (valeurs acceptées limitées — voir §14)
GET	/projects	admin-tier	Liste tous les projets
POST	/users/:userId/projects/grant	admin-tier	Octroie une permission `UserProject`
POST	/users/:userId/projects/revoke	admin-tier	Révoque une permission
GET	/users/:userId/projects	admin-tier	Projets liés à un utilisateur

12.3 `/projects` — Cœur CRM (routeur legacy + module, désambiguïsés par UUID)

Méthode	Route (extrait représentatif)	Description
GET	/, /my-projects, /myprojects, /:id, /:id/full	Listes et fiches (portée selon rôle)
POST / PUT / DELETE	/, /:id	Création / modification / suppression
PUT	/:id/move-stage, /:id/owner, /:id/status, /:id/archive, /:id/unarchive	Transitions métier (archive/unarchive = root admin uniquement)
GET/POST/PUT/DELETE	/:id/comments[/:commentId], /:id/devis[/:devisId], /:id/bondecommande[/:bdcId]	Sous-ressources documentaires et collaboratives
GET/POST/DELETE	/:id/actions, /actions/:actionId/reminders, /reminders/:id	Actions et rappels
GET	/export	Export Excel (exceljs)
GET	/kpi/*, /dashboard/*, /kpis/overview	~15 endpoints de KPI (validation, statuts, mensuel, carte, par utilisateur)
GET	/calendar, /reminders/upcoming, /nearby	Vue agenda et géoproximité
GET	/statuses, /pipeline, /missing-fields	Vue pipeline enrichie (module)

12.4 `/commercial-contacts`

Méthode	Route	Description
GET	/, /:id, /user-names/list, /calendar/relances	Listes et fiche (portée selon rôle commercial/admin)
POST	/, /select-commercial, /assign	Création, sélection de profil commercial, assignation en masse (admin)
PUT / DELETE	/:id	Modification / suppression
GET/POST	/:id/actions	Actions liées au contact
POST/PUT	/:id/relances[/:relanceId]	Planification/mise à jour des relances
GET	/kpi, /my-kpi, /analytics, /kpi/me	KPI (admin-tier + commercial)

12.5 Modules en couches (`src/modules/*`)

Préfixe	Endpoints représentatifs	Protection
/pipeline-stages	CRUD + PUT /reorder	Lecture : authRequired. Écriture : admin-tier
/pipeline (kanban)	GET /kanban, /stages, /projects ; PUT /projects/:id/stage	authRequired
/action-types	CRUD	Lecture : authRequired. Écriture : admin-tier
/dashboard	/kpis, /kpi, /professional, /user	kpiRoleGuard (sauf /user)
/admin-dashboard	GET /stats	kpiRoleGuard
/archive-requests	/my, /pending-count, POST / (ouvert) ; /, /admin, /pending, /stats, PUT /:id/approve|reject, DELETE /:id (requireRootAdmin)	Mixte (voir §11.6)
/maintenance-requests	/my, POST / (ouvert) ; /technicians, /, /stats, accept/reject/assign/start/complete/delete (requireMaintenanceManager)	Mixte (voir §11.7)
/hr-requests	création ouverte ; accept/reject/process/comment (requireHrReviewer)	Mixte (voir §11.8)
/industrial-records	CRUD (requireRole admin-tier + responsable_logistique_achat)	Portée propriétaire pour resp. logistique
/por-promesh	CRUD + /new, /operators, /dashboard, /stats, /:id/validate, /:id/pdf, /:id/attachments	Portée propriétaire pour resp. logistique
/recuperables	POST / (upsert), GET /, GET /:id, PUT /:id/terminer, DELETE /:id	Portée propriétaire pour resp. logistique

12.6 Divers (référentiels, notifications, intégrations)

Préfixe	Endpoints	Notes
/api/clients	POST /import-csv, GET /all, POST /import-factures	Table orpheline `clients` — voir §10.8
/companies, /engineers, /architects	GET /all	Référentiels dédupliqués
/crm	GET /kpi/global, /kpi/projects, /kpi/users, GET /upcoming-followups	KPI admin-tier + vue relances unifiée
/utils	GET /geocode, GET /expand-maps	Public, proxy Nominatim/OpenStreetMap
/google-calendar	/auth-url, /callback, /status, POST /disconnect, POST /webhook	/callback et /webhook publics (identité portée par un état signé ou un jeton de canal)
/metabase	GET /token	Vérification JWT indépendante (`JWT_SECRET` , pas `JWT_ACCESS_SECRET`) — voir §14
/notifications	GET /, POST /send-email (**sans authRequired**), PUT /read-all, PUT /:id/read, DELETE /:id	Anomalie de sécurité — voir §14
/tasks	CRUD, GET /my-projects	Portée créateur ou admin-tier
/users	/commercials, /me/profile, /:id/profile	Profil RH utilisé pour l'auto-remplissage des demandes
/projetactions	POST/GET /projects/:projectId/actions, /timeline	Contrôleur distinct de `modules/project-actions` (duplication)

Routes déclarées mais jamais montées dans `app.js` (code mort) : `projectComments.routes.js` , `projectTimeline.routes.js` , `upload .js` (espace dans le nom de fichier) — deux d'entre elles référencent des symboles non importés et échoueraient si elles étaient un jour montées.

13. Glossaire

Terme	Définition
Pipeline / Étape (PipelineStage)	Représentation du entonnoir de vente sous forme d'étapes ordonnées et personnalisables (remplace un ancien ENUM figé).
Relance	Rappel planifié de contact avec un prospect (contact commercial) ou de suivi d'action (projet), avec synchronisation calendrier et notifications.
Devis / Bon de commande	Documents commerciaux attachés à un projet (fichier téléversé + métadonnées), sans workflow de statut formel.
POR PROMESH	Rapport d'équipe détaillé (« Procès-verbal Opérationnel de Réception ») pour la ligne de production PROMESH, verrouillable après validation.
Récupérables	Suivi des matières/déchets récupérés en production, par machine/ligne/poste/jour.
Root Admin	Acteur unique (adresse e-mail codée en dur) seul habilité à décider des demandes d'archivage/désarchivage.
Admin-tier	Regroupement informel des rôles `admin`, `superadmin`, `superadmin2` partageant la majorité des permissions élevées.
JWT (JSON Web Token)	Jeton signé utilisé pour l'authentification sans état ; ici un jeton d'accès (15 min) et un jeton de rafraîchissement (7 j).
MFA / OTP	Authentification multi-facteurs par code à usage unique envoyé par e-mail (actuellement désactivée).
RBAC	Contrôle d'accès basé sur les rôles ; ici partiellement implémenté (rôles + e-mails codés en dur, pas de moteur de permissions configurable).
Sequelize	ORM Node.js utilisé pour modéliser les tables PostgreSQL et leurs associations.
Migration	Script versionné décrivant un changement de schéma de base de données, exécuté via `sequelize-cli`.
Agrégat (DDD)	Groupe d'entités traitées comme une unité de cohérence, avec une entité racine responsable des invariants.
Value Object (DDD)	Concept sans identité propre, défini uniquement par ses attributs (ex. : spécification produit, période de congé).
Bounded Context (DDD)	Frontière logique à l'intérieur de laquelle un modèle et un vocabulaire sont cohérents.
Cascade (ON DELETE)	Règle de suppression en chaîne : la suppression d'une ligne parente supprime automatiquement ses lignes filles.
Soft-delete (paranoid)	Suppression logique (colonne `deletedAt`) sans effacement physique de la ligne ; seule `pipeline_stages` l'utilise ici.
Webhook	Notification HTTP poussée par un système tiers (ici Google Calendar) vers le backend.
Socket.IO	Bibliothèque de communication temps réel bidirectionnelle (WebSocket + repli) utilisée pour les notifications.
Owner-scoped	Motif d'autorisation limitant la visibilité d'un rôle à ses propres enregistrements (`createdBy`).
GetX / Provider	Deux bibliothèques de gestion d'état Flutter cohabitant dans le frontend.
Dio	Client HTTP Flutter utilisé par `ApiClient` avec intercepteurs (jeton, gestion des erreurs 401).
ERD (Entity-Relationship Diagram)	Diagramme entité-association représentant les tables et leurs relations.

14. Conclusion & recommandations

14.1 Synthèse

Le CRM CBI Tunisia est un système fonctionnellement riche et cohérent : il couvre correctement le cycle commercial (prospection → projet → pipeline → archivage), les processus internes (RH, maintenance, archivage) avec un patron de workflow réutilisé, et les opérations industrielles avec des règles de gestion précises (verrouillage POR PROMESH, fenêtres temporelles Récupérables, dérivations automatiques). L'architecture technique est standard et compréhensible (Express + Sequelize + PostgreSQL + Flutter), sans dépendance exotique. Les principales faiblesses relevées sont des **dettes d'évolution** typiques d'un projet ayant grandi organiquement : deux styles d'architecture backend cohabitant, une autorisation hybride rôle/e-mail plutôt qu'un RBAC configurable, et plusieurs duplications frontend/backend qui augmentent le risque de divergence silencieuse.

14.2 Recommandations priorisées

P0 — Critique, à traiter immédiatement

> **Rotation des secrets compromis :** `Backend Master/.env.example` contient des identifiants réels et non des exemples (mot de passe SMTP, mots de passe super-administrateur, clé de chiffrement AES des jetons Google, secret MFA). Actions : (1) régénérer immédiatement tous ces secrets en production, (2) retirer le fichier de l'historique Git (réécriture d'historique ou rotation + purge), (3) remplacer son contenu par de véritables placeholders, (4) ajouter `.env*` (hors `.env.example` nettoyé) à `.gitignore` si ce n'est pas déjà le cas.

> **Endpoint non authentifié :** `POST /notifications/send-email` ne passe par aucun middleware d'authentification, contrairement à toutes les routes voisines du même fichier — un appelant anonyme peut déclencher un envoi d'e-mail et créer une notification pour un `toUserId` arbitraire. Ajouter `authRequired` immédiatement.

P1 — Élevé

- **Unifier la vérification de jeton :** consolider les trois implémentations JWT (`auth.middleware.js` , `verifyToken.js` , `moduleAccessGuard.js`) en un seul point de vérité, y compris pour l'intégration Metabase.
- **Résoudre la double route `/projects` :** le montage simultané du routeur legacy et du routeur modulaire, désambiguïsé par un test de format UUID, est fragile face à toute évolution future des deux fichiers ; migrer complètement vers le style modulaire.
- **Remplacer les e-mails codés en dur par une table de délégation :** root admin, gestionnaire RH, responsable maintenance et demandeur maintenance industrielle sont actuellement des constantes de code source ; tout changement de titulaire nécessite un redéploiement. Une table `role_delegations` (rôle fonctionnel → utilisateur actif) supprimerait ce couplage.
- **Faire fonctionner le rafraîchissement de jeton côté client :** `POST /auth/refresh` existe côté backend mais n'est jamais appelé par Flutter ; implémenter le rafraîchissement silencieux évitera des déconnexions intempestives.

P2 — Moyen

- **Dédupliquer le frontend :** converger vers un seul `ApiClient` , un seul `UserProfileService` , un seul `NotificationController` , et choisir une convention unique de gestion d'état (GetX *ou* Provider, pas les deux).
- **Généraliser le patron « Demande d'approbation » :** HR-Requests, Maintenance-Requests et Archive-Requests partagent la même forme d'automate (§8.5) mais sont implémentés indépendamment ; une abstraction commune réduirait la duplication et les risques de divergence de règles.
- **Fusionner `user_projects` et `project_members` :** deux tables de jonction M:N portant la même relation avec des vocabulaires de permission différents.
- **Nettoyer le code mort :** supprimer ou réactiver consciemment `projectComments.routes.js` , `projectTimeline.routes.js` , `upload .js` , `reminderService.js` .
- **Ajouter les contraintes FK physiques manquantes :** a minima sur `por_promesh_non_conformites.porPromeshId` et sur l'unicité de `trusted_devices(userId, deviceId)` .
- **Corriger le default hors domaine** de `commercial_contacts.typeClient` (« autre » n'appartient pas à l'ENUM).

P3 — Faible / dette technique de fond

- Uniformiser le vocabulaire de priorité (anglais vs français) entre `projects.priority` et `project_actions.priorite` / `tasks.priority` .
- Clarifier le sort de la table orpheline `clients` (l'intégrer au modèle Sequelize ou la retirer si elle n'est plus nécessaire).
- Envisager, si la charge augmente, la cible d'architecture décrite au §2.3 (Nginx, cluster Node, Redis, stockage objet) — aucune urgence tant que le volume actuel reste modéré.
- Étendre la couverture i18n réelle (de nombreux libellés restent en français codé en dur malgré 7 locales déclarées).

14.3 Mot de la fin

Ce dossier a été produit intégralement par lecture du code source, sans aucune modification : chaque diagramme, règle de gestion et anomalie cité renvoie à un fichier ou un comportement réellement présent dans `Backend Master` et `Dash Master Toolkit` au 27 juillet 2026. Il peut servir de socle à un audit de sécurité plus approfondi, à un plan de refactorisation, ou à l'onboarding de nouveaux contributeurs sur le projet.